

MEDIATEK

MT8676 Hypervisor OTA User Manual

Version: 1.0
Release date: 2024-12-27

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-12-27	Ming Ji	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	4
List of Tables	5
1 Hypervisor (L+L+A) OTA Solution Description	6
2 Hypervisor OTA Abbreviation	7
3 Overall Centralized Upgrade of Hypervisor (L+L+A)	8
3.1 Enable the Overall Centralized Upgrade Solution	8
3.2 Program Overview	8
3.2.1 Basic Functionality	8
3.3 Architecture/Process Overview	9
3.3.1 Hypervisor (L+L+A) OTA Upgrade Architecture	9
3.3.2 Hypervisor (L+L+A) otaupgrade Compilation Architecture	10
3.4 How to Compile Hypervisor (L+L+A) OTA Package	11
3.4.1 Compile hypervisor_target_files.zip	11
3.4.2 Compile Hypervisor (L+L+A) Full Upgrade Package	12
3.4.3 Compile Hypervisor (L+L+A) Incremental Upgrade Package	12
3.5 Analysis of Upgrade Package Structure	13
3.6 How to Perform Hypervisor (L+L+A) OTA Upgrade	14
3.6.1 Full Package Upgrade (Normal Mode)	14
3.6.2 Incremental Upgrade (Normal Mode)	15
3.7 How to Modify the Signature of the Upgrade Package	15
3.8 How to Remove/Add OTA Upgrade Partitions	16
3.8.1 Source of OTA Upgrade Partition	16
3.8.2 Remove OTA Upgrade Partition	16
3.8.3 Add OTA Upgrade Partition	18
3.9 Selection of Image Loading for AB system	21
3.9.1 Boot Control Information Description	21
3.9.2 Image Loading Selection Process	22
3.10 Manually Configure Boot Slot	22
3.11 System Rollback	22
3.11.1 Basic Principles of System Rollback	23
3.11.2 System Rollback Detection Process	24
3.11.3 Changes in Boot Control Parameters for System Rollback	25
3.11.4 System Rollback Test Cases	26
3.12 Power Outage Upgrade Protection	28
3.12.1 Upgrade Status Information	28
3.12.2 Resume Upgrade Inspection Process	29
4 Partial Centralized Upgrade of Hypervisor (L+L+A)	31
4.1 Enable Partial Centralized Upgrade Scheme	31
4.2 Program Overview	31
4.2.1 Basic Functionality	31
4.3 Architecture/Process Overview	32

4.3.1	Hypervisor (L+L+A) OTA Upgrade Architecture	32
4.3.2	Hypervisor (L+L+A) Upgrade Package Compilation Architecture	33
4.4	How to Compile Hypervisor OTA Upgrade Package	35
4.4.1	Compile Hypervisor Linux OS (L+L) OTA Upgrade Package	35
4.4.2	Compile the Hypervisor Android OS Upgrade Package	37
4.5	Analysis of Upgrade Package Structure	38
4.6	How to Perform Hypervisor (L+L+A) OTA Upgrade	38
4.6.1	Full Package Upgrade (Normal Mode)	38
4.6.2	Incremental Package Upgrade (Normal Mode)	39
4.7	How to Modify the Signature of the Hypervisor Linux OS (L+L) Upgrade Package	40
4.8	How to Remove/Add OTA Upgrade Partitions	41
4.8.1	Source of OTA Upgrade Partition	41
4.8.2	Remove OTA Upgrade Partition	41
4.8.3	OTA Upgrade Partition	42
4.9	Selection of Boot Slot	45
4.9.1	Boot Control Information Description	45
4.9.2	Boot Slot Selection Mechanism	47
4.10	Android Merge	48
4.11	System Rollback	49
4.11.1	Basic Principles of System Rollback	49
4.11.2	System Rollback Process	50
4.11.3	System Test Cases	51
4.12	Power Outage Upgrade Protection	53
4.12.1	Upgrade Status Information	53
4.12.2	Resume Upgrade Inspection Process	54
Attachment 1 Additional Terms		55

List of Figures

Figure 3-1.	Hypervisor (L+L+A) OTA update architecture	9
Figure 3-2.	Hypervisor (L+L+A) otapackage compilation architecture	10
Figure 3-3.	Hypervisor (L+L+A) otapackage structure	13
Figure 3-4.	payload.bin composition structure	14
Figure 3-5.	OTA upgrade partition source	16
Figure 3-6	Hypervisor (L+L+A) boot control parameters	21
Figure 3-7.	Hypervisor (L+L+A) image loading process	22
Figure 3-8.	Yocto boot failure scenario after OTA upgrade	23
Figure 3-9.	Android boot failure scenario after OTA upgrade	23
Figure 3-10.	T-box boot failure scenario after OTA upgrade	24
Figure 3-11.	Hypervisor (L+L+A) rollback detection process	24
Figure 3-12.	Yocto & Android & T-box boot status	25
Figure 3-13.	Yocto & Android & T-box boot status	25
Figure 3-14.	Yocto & Android & T-box boot status	26
Figure 3-15.	Yocto & Android & T-box boot status	26
Figure 3-16.	Upgrade status information	29
Figure 3-17.	Hypervisor resume update flow	30

Figure 3-18. Hypervisor check resume update flow	30
Figure 4-1. Hypervisor (L+L+A) OTA update architecture	32
Figure 4-2. Hypervisor L+L (Yocto + T-box) compilation otapackage architecture	33
Figure 4-3. Hypervisor Android compilation upgrade package architecture	34
Figure 4-4. OTA upgrade partition source	41
Figure 4-5. Hypervisor (L+L+A) boot control parameters	46
Figure 4-6. Hypervisor (L+L+A) boot slot selection mechanism	47
Figure 4-7. Android merge mechanism	48
Figure 4-8. Yocto boot failure scenario after OTA upgrade	49
Figure 4-9. Android boot failure scenario after OTA upgrade	49
Figure 4-10. T-box boot failure scenario after OTA upgrade	50
Figure 4-11. Hypervisor rollback flow	51
Figure 4-12. Upgrade status information	53

List of Tables

Table 2-1. Abbreviation	7
Table 3-1. otapackage component description	13
Table 3-2. payload.bin component description	14
Table 3-3. Hypervisor (L+L+A) boot control parameter description	21
Table 3-4. Upgrade status information description	29
Table 4-1. Boot control parameter description	46
Table 4-2. Upgrade status information description	53

1 Hypervisor (L+L+A) OTA Solution Description

Note: L+L+A refers to Yocto (Linux OS) + T-box (Linux OS) + Android (Android OS)

For the OTA update of Hypervisor (L+L+A), MediaTek supports the following two update schemes:

- **Solution 1:** Overall centralized upgrade, which means Yocto serves as the Host OS to unify the upgrade of Yocto + T-box + Android. For detailed introduction, please refer to [Chapter 3 Overall Centralized Upgrade of Hypervisor \(L+L+A\)](#)
- **Solution 2:** Partial centralized upgrade, which means Yocto serves as the Host OS to uniformly upgrade Yocto + T-box, while Android uses the Android Update Engine for separate upgrades. For detailed introduction, please refer to [Chapter 4 Partial Centralized Upgrade of Hypervisor \(L+L+A\)](#).

2 Hypervisor OTA Abbreviation

Table 2-1. Abbreviation

Abbreviations	Explanation
Bootctl	Boot Control A/B slot identification structure, stored in the misc partition
LK2	Little Kernel 2 Microkernel
OTA	Over-The-Air OTA download upgrade, a remote update technology

3 Overall Centralized Upgrade of Hypervisor (L+L+A)

This chapter primarily introduces the centralized upgrade scheme of Hypervisor (L+L+A) OTA Update, which includes: 1. Configuration settings to enable this scheme; 2. Overview of the scheme architecture; 3. Compilation of upgrade packages; 4. System rollback procedures. This scheme is enabled by default in the public version.

3.1 Enable the Overall Centralized Upgrade Solution

If you need to enable the centralized upgrade solution, please set the following configurations:

1. Add SUPPORT_BOOTCTRL_V3 := yes in <yocto_project_name>.mk and <tbbox_project_name>.mk in LK2

For example:

```
src/bsp/lk2/project/auto8676p1_64_hyp.mk
SUPPORT_BOOTCTRL_V3 := yes
src/bsp/lk2/project/auto8676p1_64_uos_tbox.mk
SUPPORT_BOOTCTRL_V3 := yes
```

2. Add SUPPORT_BOOTCTRL = "3" in the <yocto_project_name>.conf and <tbbox_project_name>.conf in meta.

For example:

```
meta/meta-mediatek-mt8676-hyp/conf/machine/auto8676p1_64_hyp.conf
SUPPORT_BOOTCTRL = "3"
meta/meta-mediatek-mt8676-hyp/conf/machine /auto8676p1_64_uos_tbox.conf
SUPPORT_BOOTCTRL = "3"
```

Note: Please do not support different types of OTA upgrade schemes simultaneously in a single project.mk or project.conf.

3.2 Program Overview

3.2.1 Basic Functionality

Hypervisor (L+L+A) overall centralized upgrade plan supports the following functions:

- Support seamless updates in the device background.
- Support system rollback to prevent the system from becoming inoperable after an upgrade failure.
- Support integrity and accuracy verification of the upgrade package.
- Support simultaneous upgrades of multiple OSs.
- Support full package upgrades and incremental upgrades.
- Support manual switching between A and B.
- Support power outage relay upgrades.

But it also has the following limitations:

- Updating the partition layout is not supported, the partition layout must remain consistent between versions before and after the upgrade. Subsequent modifications to the partition layout cannot be made through OTA updates.
- Mutual upgrades between AB partitions and non-AB partitions are not supported.

- The partition image that needs to be upgraded must be in xxx.img format, for example: boot.img, and other file formats are not supported.
- The partition type that requires upgrading must be a read-only (RO) partition.
- Android Virtual AB is not supported.
- OS upgrades are not supported independently.

3.3 Architecture/Process Overview

3.3.1 Hypervisor (L+L+A) OTA Upgrade Architecture

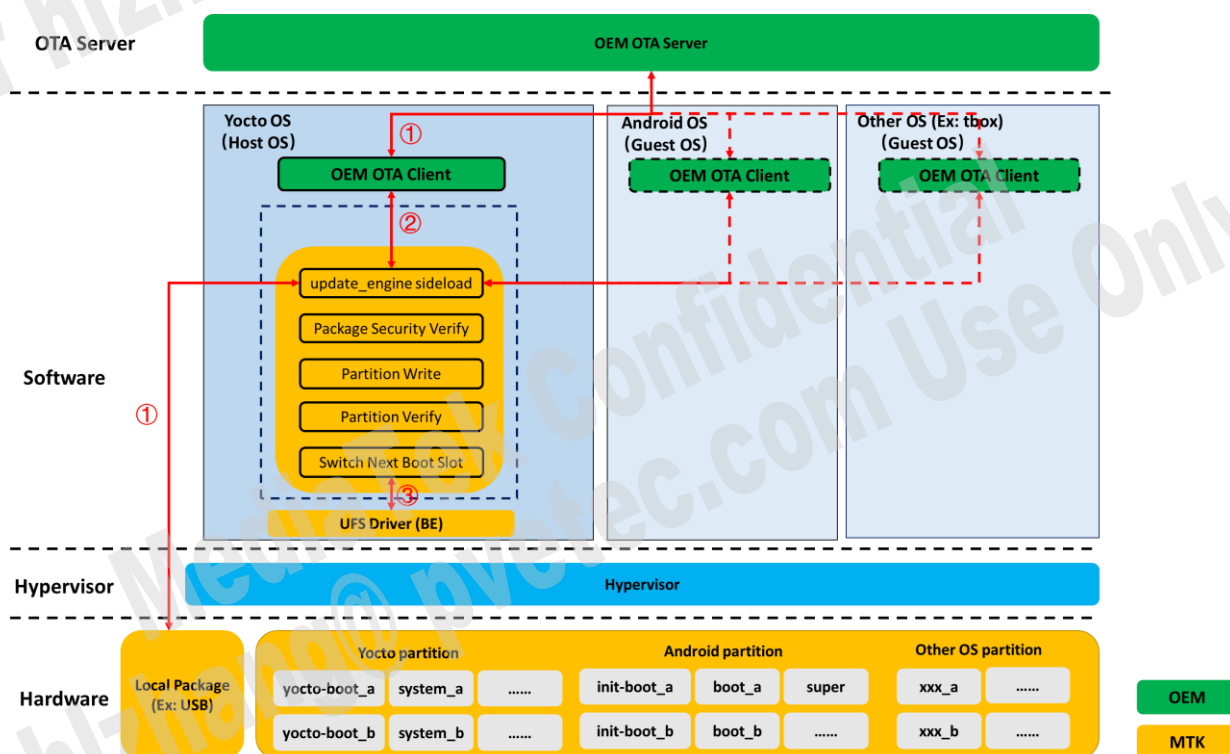


Figure 3-1. Hypervisor (L+L+A) OTA update architecture

Figure 3-1 describes the overall centralized upgrade solution architecture of the Hypervisor (L+L+A). The following information can be obtained from the figure:

1. The Hypervisor (L+L+A) OTA upgrade package supports obtaining it from the OTA Server or local storage space. MediaTek currently only supports OTA upgrades through local upgrade packages, downloading the upgrade package from the OTA Server using the network needs to be implemented by the OEM.
2. Hypervisor OTA upgrade is completed entirely on Yocto OS (Host OS), and other Guest OSs (Android OS + T-box OS) do not participate in the upgrade operation.
3. For example, completing the OTA upgrade through a local upgrade package, the general process of Hypervisor (L+L+A) OTA upgrade is as follows:
 - 1) Store the upgrade package in the local platform storage environment.
 - 2) Execute OTA update script to call update_engine sideload to trigger the upgrade.
 - 3) Use Package Security Verify to check the integrity and accuracy of the upgrade package.

- 4) Use Partition Write to write the data in the upgrade package to the corresponding partition storage space through the UFS Driver.
- 5) Use Partition Verify to perform accuracy verification on the data updated and written to the partition, ensuring the accuracy of the actual written data.
- 6) Use Switch Next Boot Slot to set the next system boot slot, ensuring that the system boots from the updated slot next time.

3.3.2 Hypervisor (L+L+A) otapackage Compilation Architecture

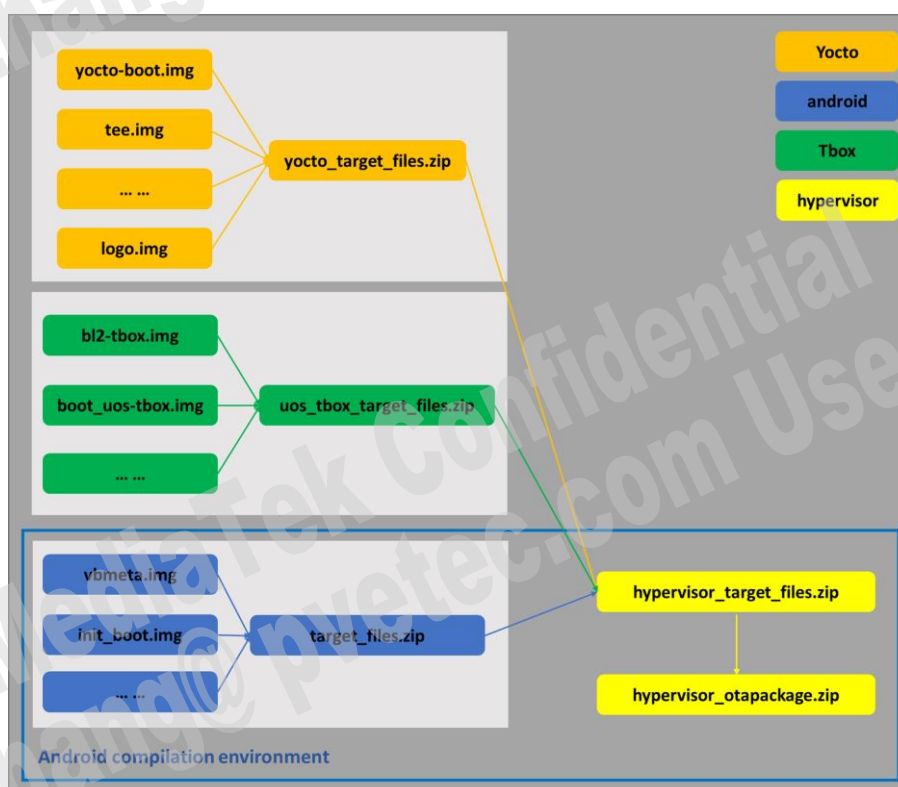


Figure 3-2. Hypervisor (L+L+A) otapackage compilation architecture

Figure 3-2 describes the architecture of the Hypervisor (L+L+A) OTA overall centralized upgrade solution to compile the upgrade package. The following information can be obtained from the figure:

1. Hypervisor (L+L+A) overall centralized upgrade only requires compiling one OTA upgrade package.
2. The Hypervisor (L+L+A) compilation upgrade package must be performed in the Hypervisor Android compilation environment.
3. The general process for compiling the hypervisor upgrade package is as follows:
 - Compile Hypervisor yocto_target_files.zip.
 - Compile Hypervisor uos_tbox_target_files.zip.
 - Compile Hypervisor Android target_files.zip.
 - Merge yocto_target_files.zip, uos_tbox_target_files.zip, and targetfiles.zip into the final hypervisor_target_files.zip.
 - Compile the full upgrade package and incremental upgrade package using hypervisor_target_files.zip in the Android build environment.

Note: For detailed instructions on compiling the Hypervisor OTA upgrade package, please refer to [Section 3.4 How to Compile Hypervisor \(L+L+A\) OTA Package](#).

3.4 How to Compile Hypervisor (L+L+A) OTA Package

3.4.1 Compile hypervisor_target_files.zip

1. Build yocto_target_files.zip

The target_files.zip of Yocto OS (Host OS) is compiled by Hypervisor_make_yocto_targetfiles.py, the script is located at:
meta/meta-mediatek-mt8676-hyp/recipes-devtools/ota-tools/files

During the Yocto OS compilation stage, the system will automatically execute this script for packaging, and the final yocto_target_files.zip is located at:

build/tmp/deploy/images/<project_name>/yocto_target_files.zip

2. Build uos_tbox_target_files.zip

The target_files.zip of T-box OS (Guest OS) is generated by Hypervisor_make_uos_tbox_targetfiles.py, the script is located at:

meta/meta-mediatek-mt8676-hyp/recipes-devtools/uos-tbox-ota-tools/files

During the T-box OS compilation stage, the system will automatically execute this script for packaging, and the final uos_tbox_target_files.zip is located at:

build/tmp/deploy/images/<project_name>/uos_tbox_target_files.zip

3. Build Android target_files.zip

The target_files.zip of Android is automatically compiled during the build Android OS phase, and the final target_files.zip is located at:

out/target/product/<project_name>/merged/ target_files.zip

4. Build hypervisor_target_files.zip

hypervisor_target_files.zip needs to use the Hypervisor_3OS_merge_targetfiles.py script to merge yocto_target_files.zip, uos_tbox_target_files.zip, and target_files.zip to produce it. Hypervisor_3OS_merge_targetfiles.py is located at:
device/mediateksample/<project_name>/Hypervisor_3OS_merge_targetfiles.py

After obtaining yocto_target_files.zip, uos_tbox_target_files.zip, and target_files.zip, execute the following command in the Android compilation environment to obtain hypervisor_target_files.zip:

```
python3 Hypervisor_3OS_merge_targetfiles.py <parameter1> <parameter2> <parameter3>
```

Note:

- Hypervisor_3OS_merge_targetfiles.py: A script to compile hypervisor_target_files.zip
- <Parameter 1>: the path of yocto_target_files.zip
- <Parameter 2>: The path of uos_tbox_target_files.zip
- <Parameter 3>: the path of target_files.zip

Output file:

A file named hypervisor_target_files.zip will be generated in the current directory. This is the target_files.zip package of Hypervisor (L+L+A).

For example:

```
python3 device/mediateksample/auto8676p1_64_bsp_vm/Hypervisor_3OS_merge_targetfiles.py
3OS_target_files/yocto_target_files.zip 3OS_target_files/uos_tbox_target_files.zip
3OS_target_files/target_files.zip
```

3.4.2 Compile Hypervisor (L+L+A) Full Upgrade Package

After compiling hypervisor_target_files.zip as referenced in [Section 3.4.1 Compile hypervisor_target_files.zip](#), compile the full upgrade package for Hypervisor (L+L+A) in the Android OS compilation environment.

Compilation instructions:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey --skip_postinstall hypervisor_target_files.zip
otapackage_full.zip
```

Note:

- ota_from_target_files: Upgrade package compilation script
- build/make/target/product/security/testkey: The key path for signing the upgrade package, MediaTek defaults to using the testkey provided by Google, your company can specify another key path using the -k parameter. To change the signing key of the upgrade package, please refer to [Section 3.7 How to Modify the Signature of the Upgrade Package](#).
- hypervisor_target_files.zip: Hypervisor (L+L+A) target files.zip
- otapackage_full.zip: The final generated Hypervisor full upgrade package, the package name can be freely specified.
- Ensure compilation of the upgrade package in the Hypervisor (L+L+A) Android OS environment. Execute “source” and “lunch” before compiling the full upgrade package, failure to do so will result in a complete compilation failure.

3.4.3 Compile Hypervisor (L+L+A) Incremental Upgrade Package

Prerequisites:

To compile the incremental upgrade package, prepare two copies of hypervisor_target_files.zip in advance: 1. The first copy is for the source version of hypervisor_target_files.zip, named source_hypervisor_target_files.zip. 2. The second copy is for the target version of hypervisor_target_files.zip, named target_hypervisor_target_files.zip.

Note: Please ensure that the base version load burned on the platform and the base version hypervisor_target_files.zip are obtained from the same compilation; otherwise, it will lead to OTA incremental upgrade failure.

Compilation instructions:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey --skip_postinstall -i
source_hypervisor_target_files.zip target_hypervisor_target_files.zip otapackage_delta.zip
```

Note:

- ota_from_target_files: Upgrade package compilation script
- build/make/target/product/security/testkey: The key path for signing the upgrade package, MediaTek defaults to using the testkey provided by Google, your company can specify another key path using the -k parameter. To change the signing key of the upgrade package, please refer to [Section 3.7 How to Modify the Signature of the Upgrade Package](#).
- source_hypervisor_target_files.zip: Source version of hypervisor_target_files.zip
- target_hypervisor_target_files.zip: Target version of hypervisor_target_files.zip

- otapackage_delta.zip: The final generated Hypervisor (L+L+A) incremental upgrade package, the package name can be specified at will.
- Ensure that the upgrade package is compiled in the Hypervisor Android build environment. Execute “source” and “lunch” before compiling the full package; otherwise, incremental package compilation will fail.

3.5 Analysis of Upgrade Package Structure

Hypervisor (L+L+A) OTA upgrade package is a compressed zip file, and its main components are as follows:

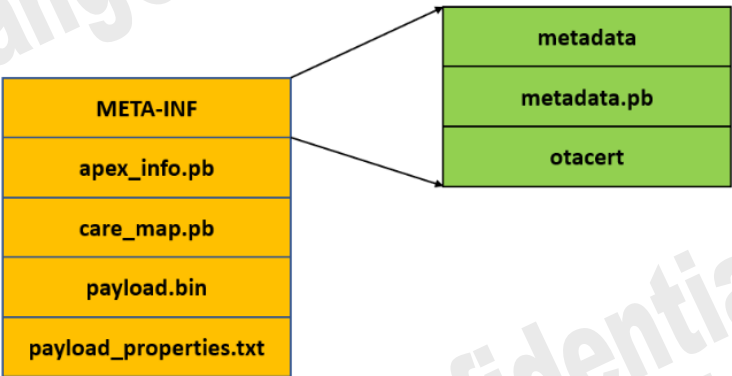


Figure 3-3. Hypervisor (L+L+A) otapackage structure

The components of the Hypervisor (L+L+A) otapackage in Figure 3-3 are described in the following table:

Table 3-1. otapackage component description

Name	Description
metadata	Store the metadata information of the upgrade package, including the upgrade version, upgrade type, etc.
metadata.pb	Store the metadata information of the upgrade package, including the upgrade version, upgrade type, etc.
otacert	Upgrade package signature verification information
apex_info.pb	Android apex module configuration information
care_map.pb	System fingerprint information
payload.bin	Upgrade image information
payload_properties.txt	The size and hash information of payload.bin and metadata

As can be seen from Table 3-1, the system upgrade information is stored in payload.bin, and its composition structure is as follows:

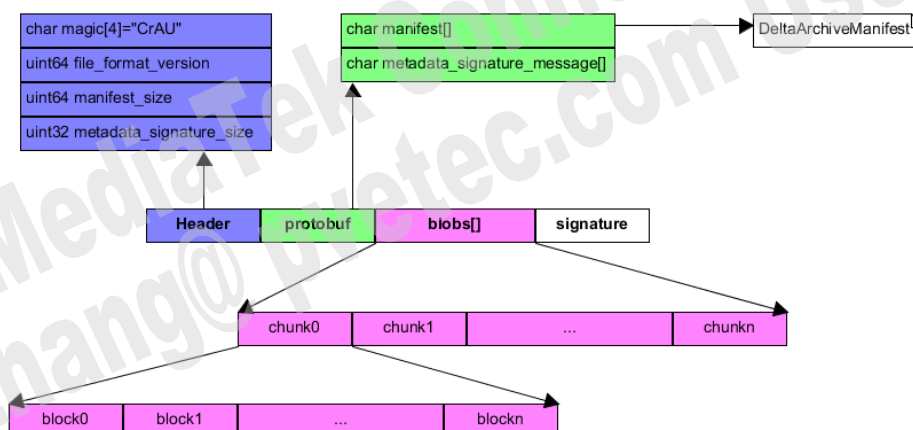


Figure 3-4. payload.bin composition structure

As can be seen from [Figure 3-4](#), the main components of payload.bin are shown in [Table 3-2](#):

Table 3-2. payload.bin component description

Name		Description
Header	Magic Number	Magic string "CrAU" identifying that this is an update payload.
	Major Version	Payload major version number.
	Manifest Size	Manifest size in bytes.
	Manifest Signature Size	Manifest signature blob size in bytes
protobuf	Manifest	The list of operations to be performed.
protobuf	Manifest Signature	The signatures of the first five fields. There could be multiple signatures if the key has changed.
blobs []	Payload Data	The list of binary blobs used by operations in the metadata.
signature	Payload Signature	The signature of the entire payload, except for the metadata signature. There could be multiple signatures if the key has changed.

3.6 How to Perform Hypervisor (L+L+A) OTA Upgrade

3.6.1 Full Package Upgrade (Normal Mode)

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get OTA package

Testing Steps:

1. Device connection Yocto adb environment
2. Execute the upgrade script:

```
python3 hypervisor_update.py --file otapackage_delta.zip > update.txt 2>&1
```

- `hypervisor_update.py`: Update script
- `otapackage_delta.zip`: OTA package, unlimited package name
- `> update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

Expected Results:

1. Update should be completed, and `update.txt` prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
 2. The first restart after the upgrade should successfully enter the HomeScreen
 3. Check whether setting-version has been updated
- Note:** If the test upgrade fails, please provide the update log and UART log in CR.

3.6.2 Incremental Upgrade (Normal Mode)

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in DUT **must be consistent with the base version provided to the OTA owner**
3. Get the OTA package from the OTA owner

Testing Steps

1. Device connection Yocto ADB environment
2. Execute the upgrade script:

```
python3 hypervisor_update.py --file otapackage_delta.zip > update.txt 2>&1
```

 - `hypervisor_update.py`: Update script
 - `otapackage_delta.zip`: OTA package, unlimited package name
 - `> update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

Expected Results:

1. Update should be completed, and `update.txt` prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
 2. The first restart after the upgrade should successfully enter the Home Screen
 3. Check whether the setting version has been updated
- Note:** If the test upgrade fails, please provide the update log and UART log in CR.

3.7 How to Modify the Signature of the Upgrade Package

During the OTA upgrade process, two keys will be used: `xxx.pk8` and `xxx.x509.pem`. The key in `xxx.pk8` format is used to sign the upgrade package during the compilation of the upgrade package, while the key in `xxx.x509.pem` format is used to verify the upgrade package during the OTA upgrade process.

If you need to use a customized key, please follow the steps below:

1. Replace the specified path of the `-k` parameter in the compilation instructions of the upgrade package in [Section 3.4.2 Compile Hypervisor \(L+L+A\) Full Upgrade Package](#) and [3.4.3 Compile Hypervisor \(L+L+A\) Incremental Upgrade Package](#) with the actual paths of `xxx.pk8` and `xxx.x509.pem`.

2. Replace the actual xxx.x509.pem file in meta/meta-mediatek/recipes-support/update-engine-sideload-u/files/ota/otacerts.zip.

3.8 How to Remove/Add OTA Upgrade Partitions

3.8.1 Source of OTA Upgrade Partition

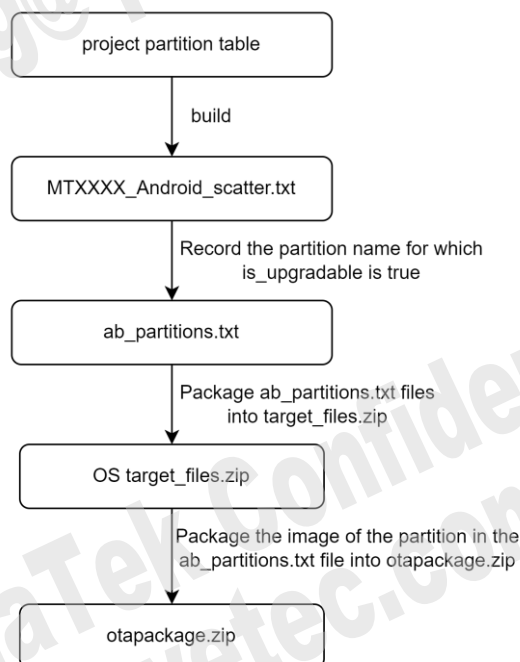


Figure 3-5. OTA upgrade partition source

The confirmation steps for the OTA upgrade partition are as follows:

1. During the compilation stage, the information in the partition table is packaged and converted into MTXXX_Android_scatter.txt, for example: MT6897_Android_scatter.txt.
2. Record the partitions in MT6897_Android_scatter.txt where is_upgradable is true to ab_partitions.txt.
3. Package ab_partitions.txt into the target_files.zip of each OS.
4. When compiling the upgrade package, the partition images recorded in ab_partitions.txt are packaged into the upgrade package.

Note: When checking the is_upgradable status of the partition, the is_upgradable of the _a partition will be prioritized. For example, when the is_upgradable of the xxxx_a partition is true and the is_upgradable of xxx_b is false, the system will still record that partition as an OTA upgrade partition.

3.8.2 Remove OTA Upgrade Partition

Note: Reducing the OTA upgrade partition will only disable the OTA upgrade for specific AB partitions and will not change the disabled AB upgrade partition to a single partition.

Hypervisor (L+L+A) includes three different OS projects, and its image composition includes: Yocto partition image + Android partition image + T-box partition image. Hypervisor (L+L+A) also includes two partition tables, one partition table is located in the Yocto codebase, the partition table contains all partition information of Yocto + Android + T-box, and this partition table is used during the compilation of Yocto and T-box; the other partition table is located in the Android codebase, the partition table only contains Android partition information, and this partition table is only used during the compilation of Android.

- **Yocto & T-box partition table:**
meta/meta-mediatek-mt8676-hyp/recipes-bsp/ptgen/files/<project_name>/partition_table_emmc_hyp_ab.csv
- **Android partition table:**
device/mediateksample/<project_name>/partition_table_emmc_hyp_ab.csv

The following are the methods for reducing OTA upgrade partitions for each OS:

Yocto OS (Host OS)

Change the OTA_Update status from Y to N in the Yocto partition table to disable the OTA upgrade partition.

For example:

- **Before modification:**

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,Y,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,Y,N,BOOTLOADERS,Y
```

- **After modification:**

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,N,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,N,N,BOOTLOADERS,Y
```

Android OS (Guest OS)

Change the OTA_Update status from Y to N in the Yocto partition table for the OTA upgrade section. No modifications are required for the Android partition table.

For example:

- **Before modification:**

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,Y,N,AUTO,Y
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

- **After modification:**

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,N,N,AUTO,Y
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

T-box OS (Guest OS)

Change the OTA_Update status from Y to N in the Tbox partition table for the section that requires disabling OTA upgrade.

For example:

- **Before modification:**

```
bl2-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-tbox.img,Y,N,AUTO,Y
bl2-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

- **After modification:**

```
bl2-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,bl2-tbox.img,N,N,AUTO,Y
```

```
bl2-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

3.8.3 Add OTA Upgrade Partition

The newly added OTA upgrade partition must meet the following prerequisites:

1. The partition must simultaneously contain <partition_name>_a & <partition_name>_b, for example: boot_a & boot_b.
2. The partition must be a read-only partition.
3. The partition image format and image name must be <partition_name>.img, for example: boot.img.

3.8.3.1 Set the Existing AB Partition as the OTA Partition

The steps for each OS to set the existing AB partitions in the partition table as OTA partitions are as follows:

Yocto OS (Host OS)

Change the OTA_Update status from N to Y in the Yocto partition table for the OTA upgrade section.

For example:

- Before modification:

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,N,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,N,N,BOOTLOADERS,Y
```

- After modification:

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,bl2.img,Y,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,bl2.img,Y,N,BOOTLOADERS,Y
```

Android OS (Guest OS)

Change the OTA_Update status from N to Y in the Yocto partition table and Android partition table under the OTA upgrade section.

For example:

- Before modification:

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,N,N,AUTO,Y
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

- After modification:

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,Y,N,AUTO,Y
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new Android OTA partition name to the ignored_partitions in Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip, preventing errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'tee', 'bl2-tbox', 'boot_uos_tbox'} >> Add the
newly added partitions to the list
```

T-box OS (Guest OS)

Change the OTA_Update status from N to Y in the T-box partition table for the section that requires disabling OTA upgrade.

For example:

- Before modification:

```
b12-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,b12-tbox.img,N,N,AUTO,Y
b12-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

- After modification:

```
b12-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,b12-tbox.img,Y,N,AUTO,Y
b12-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new T-box OTA partition name to the ignored_partitions in Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip, preventing errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'tee', 'b12-tbox', 'boot_uos_tbox'} >> Add the
newly added partitions to the list
```

Add the new T-box OTA partition name within the partitions of Hypervisor_make_uos_tbox_targetfiles.py. Ensure that the T-box compilation stage packages the partition image into uos_tbox_target_files.zip.

For example:

```
if __name__ == "__main__":
    target_path = "sos_tbox_images"
    partitions = ['b12-tbox', 'boot_uos_tbox'] >> Add the newly added partition to the
list
```

3.8.3.2 Add a Non-existent AB Partition as an OTA Partition

The steps to add the OTA partition for the AB partition that does not exist in the partition table of each OS are as follows:

Yocto OS (Host OS)

Add the corresponding AB partition in the Yocto partition table, and ensure that the partition image is <partition_name>.img, taking the addition of the customer partition as an example:

- After modification (the partition size is set to 8MB):

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

After completing the partition addition, add logic to determine the AB partition during the boot loading phase of the customer partition to ensure that the system loads into the correct partition.

Android OS (Guest OS)

Add the corresponding AB partitions in the Yocto partition table and Android partition table, and ensure that the partition image is <partition_name>.img, taking the addition of the customer partition as an example:

- After modification (the partition size is set to 8MB):

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
```

```
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new Android OTA partition name to the ignored_partitions in Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip to prevent errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'customer', 'tee', 'bl2-tbox',
'boot_uos_tbox'} >> Add the newly added partitions to the list
```

T-box OS (Guest OS)

Add the corresponding AB partition in the Yocto partition table, and ensure that the partition image is

<partition_name>.img, taking the addition of the customer partition as an example:

- After modification (the partition size is set to 8MB):

```
customer_a,Raw data,8192 ,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new Android OTA partition name to the ignored_partitions in Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip, preventing errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'tee', 'bl2-tbox', 'boot_uos_tbox',
'customer'} >> Add the newly added partitions to the list
```

Add the new T-box OTA partition name in the partitions of Hypervisor_make_uos_tbox_targetfiles.py. Ensure that the T-box compilation stage packages the partition image into uos_tbox_target_files.zip.

For example:

```
if __name__ == "__main__":
    target_path = "sos_tbox_images"
    partitions = ['bl2-tbox', 'boot_uos_tbox', 'customer'] >>Add the newly added partition
to the list
```

Note:

1. When adding a non-existent AB partition as an OTA partition, please ensure that the newly added partition is mounted as a read-only partition.
2. When adding a non-existent AB partition as an OTA partition, please ensure that the system can load and start normally after the new partition is added, and then verify the OTA upgrade.

3.9 Selection of Image Loading for AB system

3.9.1 Boot Control Information Description

The AB system contains two boot slots: slot_a and slot_b, each slot has its own independent boot control metadata. The system boot will select to load the partition image of slot_a or slot_b based on the boot control information stored in the misc partition, where the key information contained in the boot control is as follows:

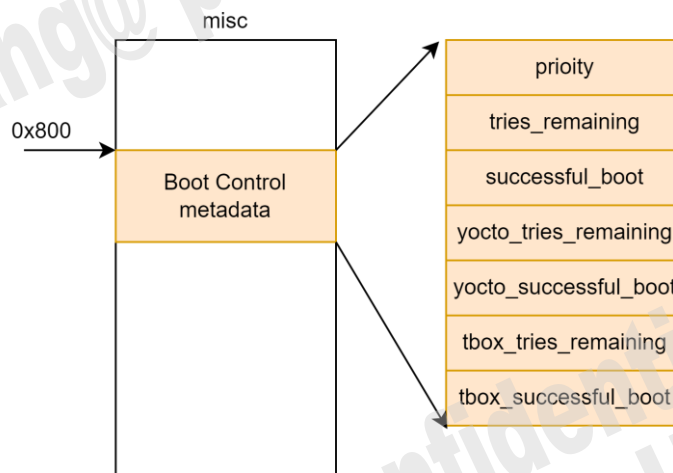


Figure 3-6 Hypervisor (L+L+A) boot control parameters

The description of each boot control parameter in Figure 3-6 is shown in Table 3-3:

Table 3-3. Hypervisor (L+L+A) boot control parameter description

Name	Description
priority	The boot priority of the slot, with a default initial value of 15, the system chooses to boot from the slot with a higher priority; if the priorities are the same, it boot from slot_a.
tries_remaining	The current remaining reboot count for the slot android is 7 by default; when the value is 0, it will trigger a system rollback.
successful_boot	Android boot success flag, 1: boot successful, 0: boot failed
yocto_tries_remaining	The remaining reboot count of the current slot yocto is 3 by default; when the value is 0, it will trigger a system rollback.
yocto_successful_boot	Yocto boot success flag, 1: boot successful, 0: boot failed
tbox_tries_remaining	The remaining restart count of the current slot T-box is 3 by default, and when the value is 0, it will trigger a system rollback.
tbox_successful_boot	T-box boot success flag, 1: boot successful, 0: boot failed

Note: Boot Control Source Code is located at: `src/bsp/lk2/platform/mediatek/common/bootctrl/bootctrl_v3`

3.9.2 Image Loading Selection Process

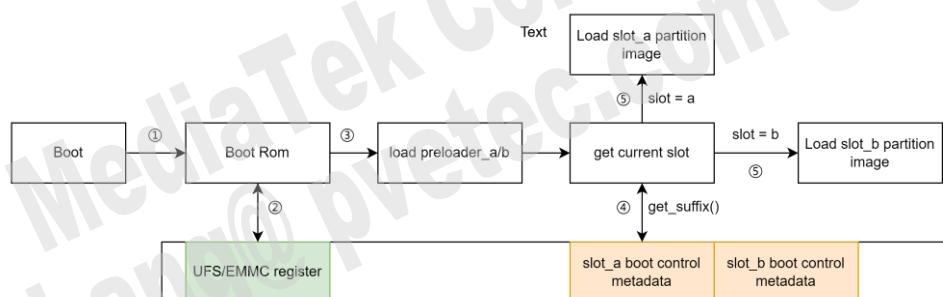


Figure 3-7. Hypervisor (L+L+A) image loading process

The image loading process started by the AB system is shown in [Figure 3-7](#):

- ① The system boot and enters the Boot ROM stage.
- ② Boot Rom will read the values of specific registers on UFS/EMMC to decide whether to load the preloader_a or preloader_b partition image.
- ③ The system boot and enters the preloader stage.
- ④ The system reads the priority values of slot_a and slot_b in the Boot Control information.
- ⑤ When the priority value of slot_a is greater than or equal to the priority value of slot_b, the system determines that the current boot slot is slot_a, and during the subsequent boot process, it will load the image in partition A; when the priority value of slot_a is less than the priority value of slot_b, the system determines that the current boot slot is slot_b, and during the subsequent boot process, it will load the image in partition B.

Note: The AB loading selection of the preloader partition is determined by the values of specific registers on UFS/eMMC, not by the slot priority of boot control metadata. For detailed information, please consult the Boot ROM & preloader owner.

3.10 Manually Configure Boot Slot

The Hypervisor (L+L+A) supports manual configuration of the boot slot using commands on the Yocto platform.

Prerequisites:

The system has successfully performed an OTA upgrade at least once, ensuring that the image information is available in both partitions AB to avoid system boot failure after switching the boot slot.

Switching command:

Shell domain	Set slot_a as the boot slot	Set slot_b as the boot slot
Yocto shell	update_engine_sideload --switch_slot=0	update_engine_sideload --switch_slot=1

3.11 System Rollback

This section mainly introduces how to roll back to the old version before the upgrade to prevent the device from becoming bricked after the Hypervisor (L+L+A) OTA upgrade is completed and the system fails to boot from the new version for the first time.

3.11.1 Basic Principles of System Rollback

Since the Hypervisor (L+L+A) Project involves multiple OS, the overall rollback plan adopted by MediaTek follows the principles below:

- Linux OS (Yocto + T-box) & Android OS overall rollback, that is, when any one OS fails to start from the new version for the first time, all OSs must roll back to the old system startup to ensure that all OSs' AB startup slots are consistent.
- Only in the scenario where the first boot from the new system fails after a successful OTA upgrade will the rollback mechanism be triggered; regular boot failures will not trigger the rollback mechanism.

Taking Yocto, T-box, and Android 3OS (L+L+A) OTA as an example, the system will have the following three rollback scenarios:

Scenario 1: Yocto OS failed to boot from the new version

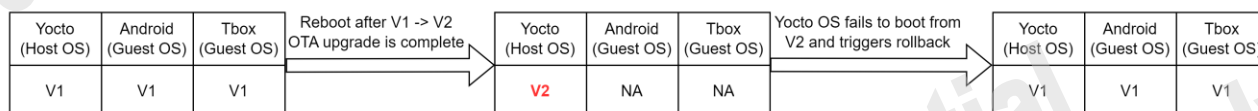


Figure 3-8. Yocto boot failure scenario after OTA upgrade

Figure 3-8 describes the version switching status of Yocto and Android after the Hypervisor (L+L+A) OTA upgrade, when Yocto fails to start from the new version and triggers a rollback. The scenario is described as follows:

- Yocto, Android, and T-box all boot from version V1.
- The system executes an OTA upgrade to version V2 and sets version V2 as the version for the next startup.
- Yocto fails to start from version V2; at this time, Android and T-box have not yet started.
- Trigger rollback. Yocto, Android, and T-box switch to version V1 for startup.

Scenario 2: Android OS failed to boot from the new version



Figure 3-9. Android boot failure scenario after OTA upgrade

Figure 3-9 describes the version switching status of Yocto, Android, and T-box after the Hypervisor (L+L+A) is upgraded from V1 -> V2 OTA. When Yocto successfully boots from the new version V2 and Android fails to boot from the new version V2, triggering a rollback, the scenario is described as follows:

- Yocto, Android, and T-box all start from version V1.
- The system executes an OTA upgrade to version V2 and sets version V2 as the version for the next startup.
- Yocto successfully boots from version V2, Android fails to boot from version V2, T-box boot status is unknown
- Trigger rollback. Yocto, Android, and T-box switch to V1 version for startup.

Scenario 3: T-box OS failed to start from the new version



Figure 3-10. T-box boot failure scenario after OTA upgrade

Figure 3-10 describes the version switching status of Yocto, Android, and T-box after the Hypervisor (L+L+A) is upgraded from V1 -> V2 OTA. When Yocto and Android successfully start from the new version V2, and T-box fails to start from the new version, triggering a rollback, the scenario is described as follows:

- Yocto, Android, and T-box all start from version V1.
- The system executes an OTA upgrade to version V2 and sets version V2 as the version for the next startup.
- Yocto and Android successfully boot from version V2; T-box fails to boot from version V2.
- Trigger rollback. Yocto, Android, and T-box switch to V1 version for startup.

3.11.2 System Rollback Detection Process

The system rollback detection process of the Hypervisor (L+L+A) is set in the LK2 stage of each OS. The detailed process is shown in Figure 3-11:

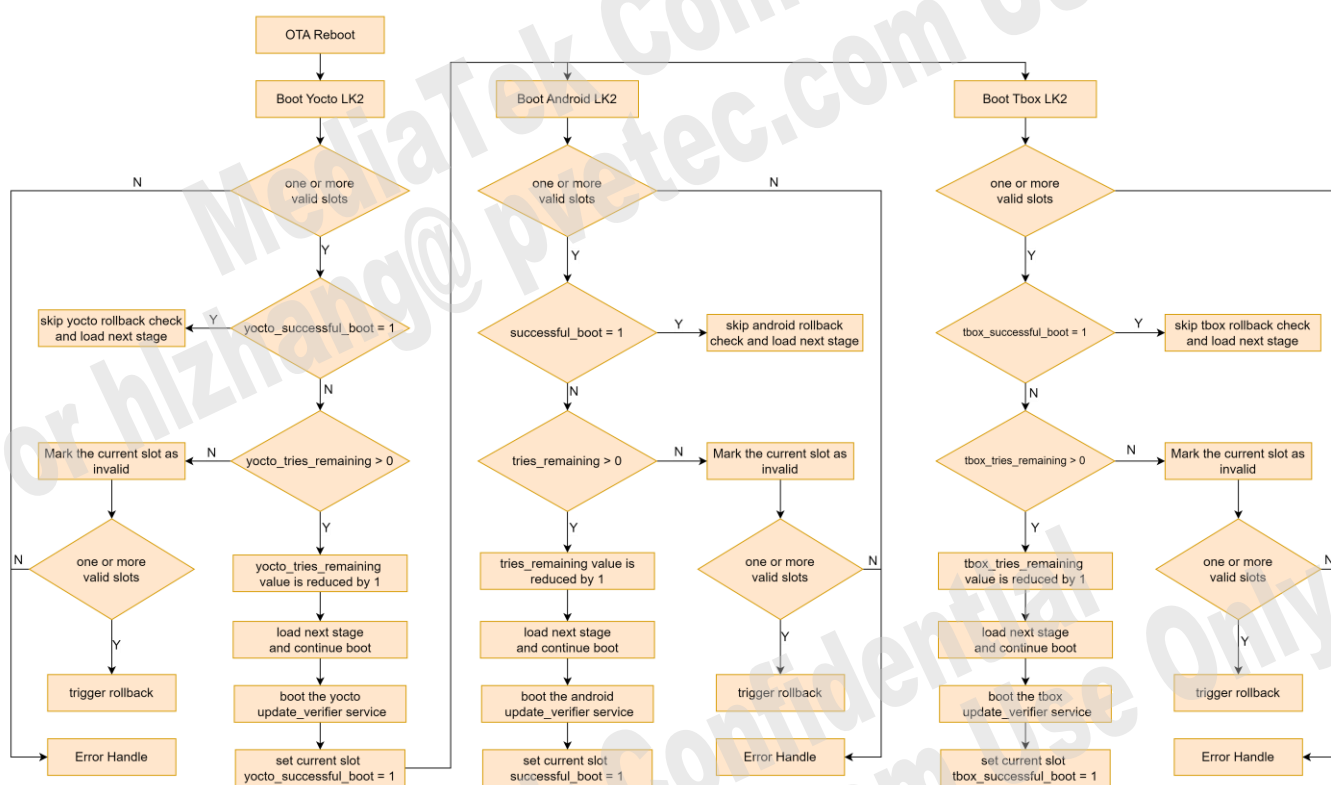


Figure 3-11. Hypervisor (L+L+A) rollback detection process

3.11.3 Changes in Boot Control Parameters for System Rollback

As shown in Figure 3-11, the essence of the system rollback mechanism is to repeatedly judge and adjust the Boot Control information, and only by triggering repeated system restarts to reduce the tries_remaining value of a system to 0 can the overall system rollback be triggered. The startup of Hypervisor (L+L+A) 3OS can be divided into the following four scenarios:

Scenario 1: Yocto & Android & T-box all boot successfully (reboot after OTA upgrade)

Yocto	Android	Tbox
OK	OK	OK

Figure 3-12. Yocto & Android & T-box boot status

The changes in the boot control parameter values at each stage are as follows:

		Normal boot			No reboot after OTA upgrade			Yocto reboot successfully after OTA upgrade			Android reboot successfully after OTA upgrade			Tbox reboot successfully after OTA upgrade	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15
yocto	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1
	yocto_successful_boot	1	0		1	0		1	1		1	1		1	1
android	tries_remaining	7	7		1	7		1	7		1	1		1	1
	successful_boot	1	0		1	0		1	0		1	1		1	1
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	3		1	1
	tbox_successful_boot	1	0		1	0		1	0		1	0		1	1

Scenario 2: Yocto & Android boot successfully, T-box fails to boot (reboot after OTA upgrade)

Yocto	Android	Tbox
OK	OK	NG

Figure 3-13. Yocto & Android & T-box boot status

The changes in the boot control parameter values at each stage are as follows:

		Normal boot			No reboot after OTA upgrade			Yocto reboot successfully after OTA upgrade			Android reboot successfully after OTA upgrade			Tbox reboot fail after OTA upgrade			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		15	14
yocto	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1		1	0
	yocto_successful_boot	1	0		1	0		1	1		1	1		1	1		1	0
android	tries_remaining	7	7		1	7		1	7		1	1		1	1		1	0
	successful_boot	1	0		1	0		1	0		1	1		1	1		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	3		1	0		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	0		1	0		1	0

Scenario 3: Yocto & T-box boot successfully, Android fails to boot (reboot after OTA upgrade)

Yocto	Android	Tbox
OK	NG	OK

Figure 3-14. Yocto & Android & T-box boot status

The changes in the boot control parameter values at each stage are as follows:

		Normal boot			No reboot after OTA upgrade			Yocto reboot successfully after OTA upgrade			Tbox reboot successfully after OTA upgrade			Android reboot fail after OTA upgrade			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		14	15		14	15		15	14
yocto	yocto_tries_remaining	3	3		1	3		1	1		1	1		1	1		1	0
	yocto_successful_boot	1	0		1	0		1	1		1	1		1	1		1	0
android	tries_remaining	7	7		1	7		1	7		1	7		1	0		1	0
	successful_boot	1	0		1	0		1	0		1	0		1	0		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	1		1	1		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	1		1	1		1	0

Scenario 4: Yocto boot failed, Android & T-box have not boot yet (reboot after OTA upgrade)

Yocto	Android	Tbox
NG	NA	NA

Figure 3-15. Yocto & Android & T-box boot status

The changes in the boot control parameter values at each stage are as follows:

		Normal boot			No reboot after OTA upgrade			Yocto reboot fail after OTA upgrade			Triggering system rollback	
		slot a	slot b		slot a	slot b		slot a	slot b		slot a	slot b
	priority	15	15		14	15		14	15		15	14
yocto	yocto_tries_remaining	3	3		1	3		1	0		1	0
	yocto_successful_boot	1	0		1	0		1	0		1	0
android	tries_remaining	7	7		1	7		1	7		1	0
	successful_boot	1	0		1	0		1	0		1	0
tbox	tbox_tries_remaining	3	3		1	3		1	3		1	0
	tbox_successful_boot	1	0		1	0		1	0		1	0

3.11.4 System Rollback Test Cases

3.11.4.1 Yocto OS Boot Failed

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get the OTA package from the OTA owner

Testing Steps:

1. Device connection Yocto ADB environment
2. Execute the upgrade script:

```
python3 hypervisor_update.py --file otapackage_full.zip > update.txt 2>&1
```
3. Check whether the upgrade has been completed successfully, update.txt contains:
[INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
4. Execute in the Yocto ADB environment:
 - adb shell
 - blockdev --setrw /dev/disk/by-partlabel/yocto-boot_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/yocto-boot_b bs=1M count=1
 - reboot

Note:

- **hypervisor_update.py**: Update script
- **otapackage_full.zip**: OTA package, unlimited package name
- **> update.txt 2>&1**: Store the upgrade log in update.txt in the current directory

Expected Results:

1. Update should be completed, and **update.txt** prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
 2. The system triggers the rollback mechanism and boots to the Home Screen
 3. Setting version should be the same as before the upgrade
- Note:** If the test upgrade fails, please provide the update log and UART log in CR

3.11.4.2 Android OS Boot Failed

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get the OTA package from the OTA owner

Testing Steps

1. Device connection Yocto ADB environment
2. Execute the upgrade script:

```
python3 hypervisor_update.py --file otapackage_full.zip > update.txt 2>&1
```
3. Check whether the upgrade has been completed successfully, update.txt contains:
[INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
4. Execute in the Yocto ADB environment:
 - adb shell
 - blockdev --setrw /dev/disk/by-partlabel/init_boot_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/init_boot_b bs=1M count=1
 - reboot

Note:

- **hypervisor_update.py**: Update script
- **otapackage_full.zip**: OTA package, unlimited package name
- **> update.txt 2>&1**: Store the upgrade log in update.txt in the current directory

Expected Results:

1. Update should be completed, and **update.txt** prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
2. The system triggers the rollback mechanism and boots to the Home Screen
3. Setting version should be the same as before the upgrade
Note: If the test upgrade fails, please provide the update log and UART log in CR.

3.11.4.3 T-box OS Boot Failed

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get the OTA package from the OTA owner

Testing Steps:

1. Device connection Yocto ADB environment
2. Execute the upgrade script:
`python3 hypervisor_update.py --file otapackage_full.zip > update.txt 2>&1`
3. Check whether the upgrade has been completed successfully, update.txt contains:
[INFO:update_attempter_android.cc(600)] **Update successfully applied, waiting to reboot.**
4. Execute in the Yocto ADB environment:
 - adb shell
 - blockdev --setrw /dev/disk/by-partlabel/boot_uos_tbox_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/boot_uos_tbox_b bs=1M count=1
 - reboot

Note:

- **hypervisor_update.py**: Update script
- **otapackage_full.zip**: OTA package, unlimited package name
- `> update.txt 2>&1`: Store the upgrade log in update.txt in the current directory

Expected Results:

1. Update should be completed, and **update.txt** prints the following log: [INFO:update_attempter_android.cc(600)]
Update successfully applied, waiting to reboot.
2. The system triggers the rollback mechanism and boots to the Home Screen
3. Setting version should be the same as before the upgrade
Note: If the test upgrade fails, please provide the update log and UART log in CR.

3.12 Power Outage Upgrade Protection

3.12.1 Upgrade Status Information

Since the device may be interrupted due to abnormal power failure during the OTA upgrade process, in order to ensure that the device can continue to complete the upgrade from the interruption point after power failure and restart during the OTA upgrade process, the system needs to save the upgrade status information in real time. The Hypervisor (L+L+A)

overall centralized upgrade solution saves the upgrade status information in the `data/misc/update_engine/prefs` directory of the yocto-userdata partition by default. The stored information is shown in [Figure 3-16](#):

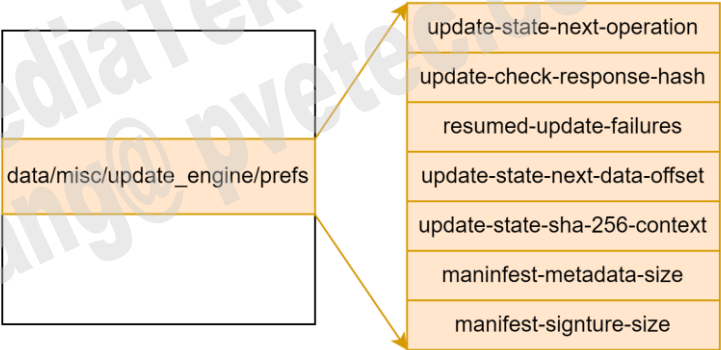


Figure 3-16. Upgrade status information

The description of each upgrade status information in the prefs directory is shown in [Table 3-4](#):

Table 3-4. Upgrade status information description

Prefs name	Prefs value	Description
kPrefsUpdateStateNextOperation	update-state-next-operation	Record the next operation that needs to be performed.
kPrefsUpdateCheckResponseHash	update-check-response-hash	Record the hash value of the last upgrade package.
kPrefsResumedUpdateFailures	resumed-update-failures	Record the number of times the upgrade is interrupted. By default, an upgrade can be interrupted a maximum of 10 times. If it exceeds 10 times, the system will force a complete re-upgrade.
kPrefsUpdateStateNextDataOffset	update-state-next-data-offset	Record the offset address of the data downloaded from the upgrade package next time.
kPrefsUpdateStateSHA256Context	update-state-sha-256-context	Record the hash value of the upgrade status information
kPrefsManifestMetadataSize	manifest-metadata-size	Record the size of metadata data
kPrefsManifestSignatureSize	manifest-signature-size	Record the size of the signature data

3.12.2 Resume Upgrade Inspection Process

Before the OTA upgrade, the system will determine whether the current upgrade is a new update or a resume update. The information for this detection comes from the data saved in the prefs directory. The detection process is shown in [Figure 3-17](#):

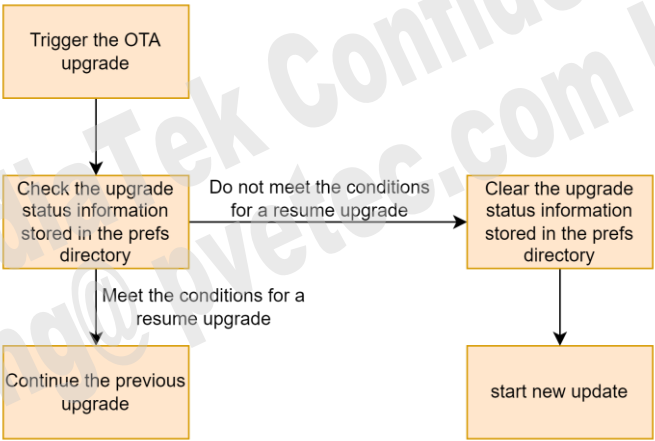


Figure 3-17. Hypervisor resume update flow

The detailed process of checking whether the information stored in the prefs directory meets the resume upgrade conditions is shown in Figure 3-18:

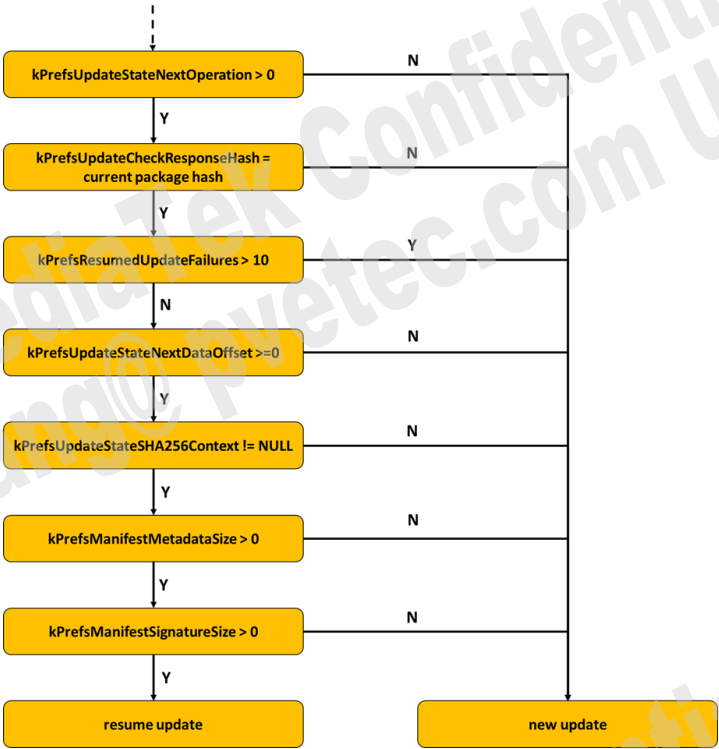


Figure 3-18. Hypervisor check resume update flow

As can be seen from Figure 3-18, only when the prefs information meets all the conditions will the system recognize this upgrade as a resume update; if any of the conditions are not met, the system will restart the upgrade and clear the previously saved upgrade status information.

4 Partial Centralized Upgrade of Hypervisor (L+L+A)

This chapter primarily introduces the Hypervisor (L+L+A) OTA update centralized upgrade solution. It includes configuration settings for enabling this solution, an overview of the solution architecture, upgrade package compilation, and system rollback. This public version does not support the solution by default and is provided for customer reference only.

4.1 Enable Partial Centralized Upgrade Scheme

If you need to enable the centralized upgrade plan, please set the following configurations:

1. Add `SUPPORT_BOOTCTRL_V1 := yes` in `<yocto_project_name>.mk` and `<tbox_project_name>.mk` of LK2

For example:

```
src/bsp/lk2/project/auto8676p1_64_hyp_sos.mk
SUPPORT_BOOTCTRL_V1 := yes
src/bsp/lk2/project/auto8676p1_64_uos_tbox.mk
SUPPORT_BOOTCTRL_V1 := yes
```

2. Add `SUPPORT_BOOTCTRL = "1"` in the `<yocto_project_name>.conf` and `<tbox_project_name>.conf` in meta.

For example:

```
meta/meta-mediatek-mt8676-hyp/conf/machine/auto8676p1_64_hyp_sos.conf
SUPPORT_BOOTCTRL = "1"
meta/meta-mediatek-mt8676-hyp/conf/machine /auto8676p1_64_uos_tbox.conf
SUPPORT_BOOTCTRL = "1"
```

Note: Please do not support different types of OTA upgrade schemes simultaneously in a single project.mk or project.conf.

4.2 Program Overview

4.2.1 Basic Functionality

Hypervisor (L+L+A) partial centralized upgrade plan supports the following functions:

- Support seamless updates in the device background.
- Support system rollback to prevent the system from becoming inoperable after an upgrade failure.
- Support integrity and accuracy verification of the upgrade package.
- Support Android & Linux independent upgrades, i.e. Yocto upgrades Yocto and T-box, Android upgrades Android
- Support full package upgrades and incremental upgrades.
- Support Android Virtual AB.
- Support power outage relay upgrades.

But it also has the following limitations:

- Updating the partition layout is not supported; the partition layout must remain consistent between versions before and after the upgrade. Subsequent modifications to the partition layout cannot be made through OTA updates.
- Mutual upgrades between AB partitions and non-AB partitions are not supported.

- The partition image that needs to be upgraded must be in xxx.img format, for example: boot.img, and other file formats are not supported.
- The partition type that requires upgrading must be a read-only (RO) partition.
- Manual switching between A and B is not supported.
- OS upgrades are not supported independently.

4.3 Architecture/Process Overview

4.3.1 Hypervisor (L+L+A) OTA Upgrade Architecture

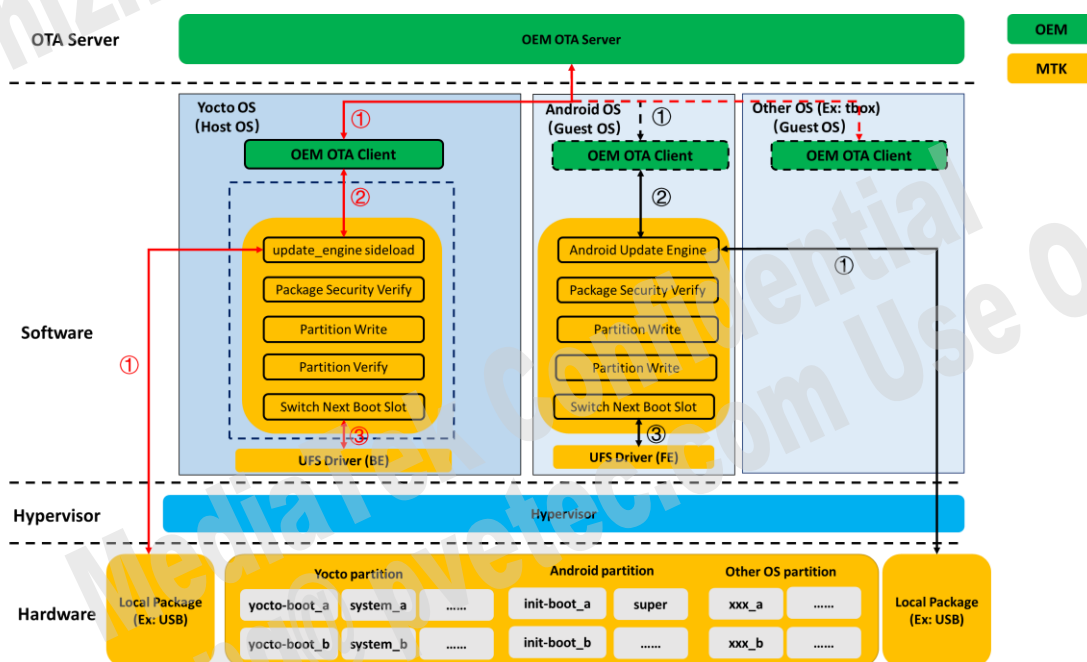


Figure 4-1. Hypervisor (L+L+A) OTA update architecture

Figure 4-1 describes the architecture of the centralized upgrade solution for the Hypervisor (L+L+A). The following information can be obtained from the figure:

1. The Hypervisor (L+L+A) OTA upgrade package supports obtaining it from the OTA Server or local storage space. MediaTek currently only supports OTA upgrades through local upgrade packages; downloading the upgrade package from the OTA Server using the network needs to be implemented by the OEM.
2. Hypervisor (L+L+A) OTA upgrade is divided into two parts: Linux and Android. The upgrade of the Linux OS is completed entirely on Yocto OS (Host OS), and T-box (Guest OS) does not participate in the upgrade operation; the upgrade of Android OS (Guest OS) is completed by Android OS. The purpose of independent upgrades for Linux and Android is to support Android Virtual AB (VAB) functionality.
3. Since the upgrades of Linux OS and Android OS are independent of each other, taking local upgrades as an example, the general process of Hypervisor (L+L+A) OTA upgrades is as follows:

Linux OS upgrade:

- 1) Store the upgrade package in the local platform storage environment.
- 2) Execute OTA update script to call **update_engine sideload** to trigger the upgrade.
- 3) Use **Package Security Verify** to check the integrity and accuracy of the upgrade package.

- 4) Use Partition Write to write the data in the upgrade package to the corresponding partition storage space through the UFS Driver.
- 5) Use Partition Verify to perform accuracy verification on the data updated and written to the partition, ensuring the accuracy of the actual written data.
- 6) Use Switch Next Boot Slot to set the next system boot slot, ensuring that the system boots from the updated slot next time.

Android OS upgrade:

- 1) Store the upgrade package in the local platform storage environment.
- 2) Execute OTA update script to call android update_engine to trigger the upgrade.
- 3) Use Package Security Verify to check the integrity and accuracy of the upgrade package.
- 4) Use Partition Write to write the data in the upgrade package to the corresponding partition storage space through the UFS Driver.
- 5) Use Partition Verify to perform accuracy verification on the data updated and written to the partition, ensuring the accuracy of the actual written data.
- 6) Use Switch Next Boot Slot to set the next system boot slot, ensuring that the system boots from the updated slot next time.

4.3.2 Hypervisor (L+L+A) Upgrade Package Compilation Architecture

The Hypervisor (L+L+A) upgrade requires compiling two upgrade packages: one for upgrading L+L (Yocto + T-box) and another for upgrading A (Android). The compilation of both upgrade packages must be completed in the Android environment.

4.3.2.1 Compilation of the Hypervisor L+L (Yocto + T-box) otapackage

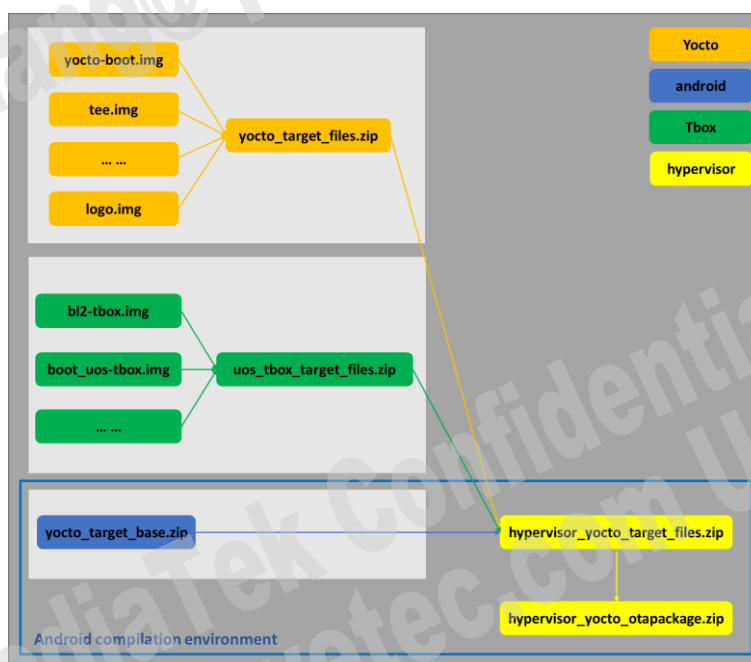


Figure 4-2. Hypervisor L+L (Yocto + T-box) compilation otapackage architecture

Figure 4-2 describes the architecture of the Hypervisor L+L (Yocto + T-box) OTA compilation upgrade package. The following information can be obtained from the figure:

1. Each Linux OS generates a `target_files.zip` during the compilation stage, used to package the partition image files that need to be upgraded for each OS.
2. In the Android OS compilation environment, a `yocto_target_base.zip` has been prepared in advance to ensure that the final `hypervisor_yocto_target_files.zip` conforms to the directory structure of the input `target_files.zip` of the Android native compilation script `ota_form_target_files.py`.
3. The upgrade package for Hypervisor L+L (Yocto + T-box) must be performed in the compilation environment of Hypervisor Android, because it is necessary to use the relevant Android compilation tools.
4. The general process of compiling the upgrade package for Hypervisor L+L (Yocto + T-box) is as follows:
 - Compile each Linux OS project to obtain the respective OS's `target_files.zip`
For example: `yocto_target_files.zip` (Yocto OS), `uos_tbox_target_files.zip` (T-box OS)
 - Merge all Linux OS `target_files.zip` and `yocto_target_base.zip` into the final `hypervisor_yocto_target_files.zip`
 - Use `hypervisor_yocto_target_files.zip` to compile and generate the full upgrade package and incremental upgrade package of Linux OS in the Android compilation environment.

Note: For detailed instructions on compiling the Hypervisor L+L (Yocto + T-box) OTA upgrade package, please refer to [Section 4.4.1 Compile Hypervisor Linux OS \(L+L\) OTA Upgrade Package](#).

4.3.2.2 Hypervisor Android Upgrade Package Compilation

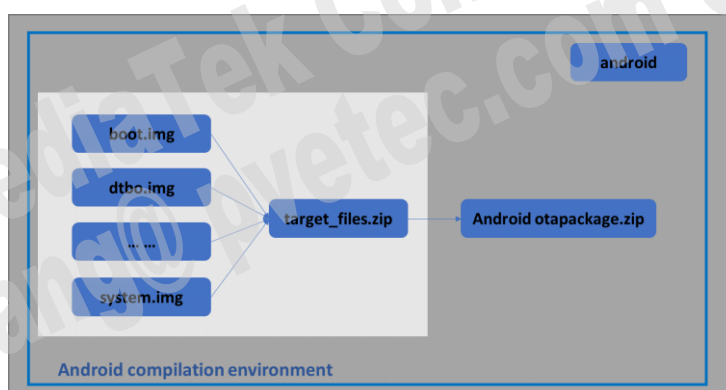


Figure 4-3. Hypervisor Android compilation upgrade package architecture

Figure 4-3 describes the architecture of the Hypervisor Android OTA compilation upgrade package. The following information can be obtained from the figure:

1. Android OS generates a `target_files.zip` during the compilation stage, which is used to package the partition images that need to be upgraded for Android OS.
2. The general process for compiling the Hypervisor Android upgrade package is as follows:
 - Compile the Android OS project to obtain the Android OS `target_files.zip`
For example: `target_file.zip` (Android OS)
 - Use Android `target_files.zip` to compile and generate a full upgrade package and a incremental upgrade package for the Android OS in the Android compilation environment.

Note: For detailed instructions on compiling the Hypervisor Android OTA upgrade package, please refer to [Section 4.4.2 Compile the Hypervisor Android OS Upgrade Package](#).

4.4 How to Compile Hypervisor OTA Upgrade Package

4.4.1 Compile Hypervisor Linux OS (L+L) OTA Upgrade Package

4.4.1.1 Compile Linux OS target_files.zip

1. Build Yocto OS target_files.zip

The target_files.zip of Yocto OS (Host OS) is generated by Hypervisor_make_yocto_targetfiles.py, the script is located at:
meta/meta-mediatek-mt8676-hyp/recipes-devtools/ota-tools/files

During the compilation stage of Yocto OS, the system will automatically execute this script for packaging, and the final yocto_targetfile.zip is located at:

build/tmp/deploy/images/<project_name>/yocto_target_files.zip

2. Build T-box OS target_files.zip

The target_files.zip of Tbox OS (Guest OS) is generated by Hypervisor_make_uos_tbox_targetfiles.py, the script is located at:

meta/meta-mediatek-mt8676-hyp/recipes-devtools/uos-tbox-ota-tools/files

During the compilation phase of T-box OS, the system will automatically execute this script for packaging, and the final uos_tbox_target_files.zip is located at:

build/tmp/deploy/images/<project_name>/uos_tbox_target_files.zip

3. Build hypervisor_yocto_target_files.zip

After compiling to obtain Yocto OS target_files.zip and Tbox OS target_files.zip, please execute the following command in the Android compilation environment:

```
python3 Hypervisor_yocto_merge_targetfiles.py <parameter1> <parameter2> <parameter3>
```

Note:

- Hypervisor_yocto_merge_targetfiles.py: Script to compile hypervisor_yocto_target_files.zip
- <Parameter 1>: The path of Yocto OS target_files.zip
- <Parameter 2>: The path of T-box OS target_files.zip
- <Parameter 3>: the path of yocto_target_base.zip

Output file:

A file named hypervisor_yocto_target_files.zip will be generated in the current directory. This file represents the final target_files.zip package of the Linux OS.

For example:

```
python3
device/mediateksample/auto8676p1_64_bsp_vm_tbox/Hypervisor_yocto_merge_targetfiles.py
tbox_target_file/yocto_target_files.zip
tbox_target_file/uos_tbox_target_files.zip
device/mediateksample/auto8676p1_64_bsp_vm_tbox/yocto_target_base.zip
```

4.4.1.2 Compile Linux OS OTA Upgrade Package

After compiling `hypervisor_yocto_target_files.zip` in [Section 4.4.1.1 Compile Linux OS target_files.zip](#), you can compile the upgrade package of Linux OS (L+L) in the compilation environment of Hypervisor Android OS.

1. Build full otapackage

Compilation instructions:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey --skip_postinstall
hypervisor_yocto_target_files.zip otapackage_yocto_full.zip
```

Note:

- `ota_from_target_files`: Upgrade package compilation script
- `build/make/target/product/security/testkey`: The key path for signing the upgrade package, MediaTek defaults to using the testkey provided by Google, your company can specify another key path, use the `-k` parameter to specify.
- `hypervisor_yocto_target_files.zip`: Hypervisor Linux OS (L+L) `target_files.zip`
- `otapackage_yocto_full.zip`: The final generated Hypervisor Linux OS (L+L) full upgrade package, the package name can be specified arbitrarily.
- Ensure compilation of the upgrade package in the Hypervisor Android build environment. Execute “source” and “lunch” before compiling the full package; otherwise, full package compilation will fail.

2. Build incremental otapackage

Prerequisites:

To compile the incremental package, prepare two copies of `hypervisor_yocto_target_files.zip` in advance: The first copy is for the base version (source version) of `hypervisor_yocto_target_files.zip`, named `source_hypervisor_yocto_target_files.zip`; The second copy is for the target version of `hypervisor_yocto_target_files.zip`, named `target_hypervisor_yocto_target_files.zip`.

Note: Please ensure that the base version load burned on the platform and the base version `hypervisor_yocto_target_files.zip` are obtained from the same compilation; otherwise, it will lead to OTA incremental upgrade failure.

Compilation instructions:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -k
build/make/target/product/security/testkey --skip_postinstall -i
source_hypervisor_yocto_target_files.zip target_hypervisor_yocto_target_files.zip
otapackage_yocto_delta.zip
```

Note:

- `ota_from_target_files`: Upgrade package compilation script
- `build/make/target/product/security/testkey`: The key path for signing the upgrade package, MediaTek defaults to using the testkey provided by Google. Your company can specify another key path by using the `-k` parameter.
- `source_hypervisor_yocto_target_files.zip`: the source version of `hypervisor_yocto_target_files.zip`
- `target_hypervisor_yocto_target_files.zip`: target version of `hypervisor_yocto_target_files.zip`
- `otapackage_yocto_delta.zip`: The final generated Hypervisor Linux OS (L+L) incremental upgrade package, the package name can be specified at will.
- Ensure that the upgrade package is compiled in the Hypervisor Android build environment. Execute “source” and “lunch” before compiling the full package; otherwise, incremental package compilation will fail.

4.4.2 Compile the Hypervisor Android OS Upgrade Package

4.4.2.1 Compile Android OS target_files.zip

The target_files.zip of Android OS is automatically generated when compiling the Android project using split build, and it is located at:

```
out/target/product/<project_name>/merged/target_files.zip
```

4.4.2.2 Compile the Android OS Upgrade Package

After compiling Android target_files.zip in [Section 4.4.2.1 Compile Android OS target_files.zip](#), you can compile the Android OS upgrade package in the Hypervisor Android OS compilation environment.

1. Build full otapackage

Compilation instructions:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 target_files.zip otapackage_android_full.zip
```

Note:

- ota_from_target_files: Upgrade package compilation script
- target_files.zip: Hypervisor Android OS target_files.zip
- otapackage_android_full.zip: The final generated Hypervisor Android OS (L+L) full upgrade package, the package name can be specified arbitrarily.
- Ensure compilation of the upgrade package in the Hypervisor Android build environment. Execute “source” and “lunch” before compiling the full package to prevent compilation failure.

2. Build incremental otapackage

Prerequisites:

To compile the incremental package, prepare two copies of the Android OS target_files.zip in advance: The base version (source version) target_files.zip, named source_target_files.zip; and the target version of target_files.zip, named target_target_files.zip.

Note: Please ensure that the base version load burned on the platform and the base version target_files.zip are obtained from the same compilation; otherwise, it will lead to OTA incremental upgrade failure.

Compilation instructions:

```
./out_sys/host/linux-x86/bin/ota_from_target_files -v --block -p out_sys/host/linux-x86 -i source_target_files.zip target_target_files.zip otapackage_android_delta.zip
```

Note:

- ota_from_target_files: Upgrade package compilation script
- source_target_files.zip: Source version of Android target_files.zip
- target_target_files.zip: Target version of Android target_files.zip
- otapackage_android_delta.zip: The final generated Android OS incremental upgrade package, and the package name can be specified arbitrarily.

4.5 Analysis of Upgrade Package Structure

Please refer to the description in [Section 3.5 Analysis of Upgrade Package Structure](#).

4.6 How to Perform Hypervisor (L+L+A) OTA Upgrade

4.6.1 Full Package Upgrade (Normal Mode)

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get OTA package (including Android OS package and Linux OS package)

Testing Steps:

Note: Please complete the Android OS upgrade first, and then perform the Linux OS upgrade to avoid system startup failure caused by mismatched slots of each OS when the platform powers off and restarts after completing an OS upgrade.

Update Android OS:

1. The device connects to the adb environment, defaulting to connect to Android adb.
2. Execute the following command to connect to Yocto adb
 - adb forward tcp:7777 tcp:6666
 - adb connect 127.0.0.1:7777
3. Execute adb devices to check whether adb is connected successfully. The correct connection is as follows:
List of devices attached
0123456789ABCDEF device >> Android device
127.0.0.1:7777 device >> Yocto device
4. Execute the Android OS upgrade script: `python3 hypervisor_android_update.py -s 0123456789ABCDEF -file otapackage_android_full.zip > update_android_full.txt 2>&1`
 - hypervisor_android_update.py: Android OS upgrade script
 - 0123456789ABCDEF: Android OS device adb number
 - otapackage_android_full.zip: Android OS OTA full upgrade package, no restrictions on the upgrade package name
 - update_android_full.txt: Android OS Upgrade Log

Update Linux OS:

1. The device connects to the adb environment, defaulting to connect to Android adb.
2. Check whether Linux adb is connected; if not connected, please execute the following command to connect Yocto adb.
 - adb forward tcp:7777 tcp:6666
 - adb connect 127.0.0.1:7777
3. Execute adb devices to check whether adb is connected successfully. A correct connection is as follows:
List of devices attached
0123456789ABCDEF device >> Android device
127.0.0.1:7777 device >> Yocto device
4. Execute the Linux OS upgrade script: `python3 hypervisor_yocto_update.py -s 127.0.0.1:7777 --file otapackage_yocto_full.zip > update_yocto_full.txt 2>&1`
 - hypervisor_yocto_update.py: Linux OS upgrade script
 - 127.0.0.1:7777: Linux OS device adb number

- otapackage_yocto_full.zip: Linux OS OTA full upgrade package, no restrictions on the upgrade package name
 - update_yocto_full.txt: Linux OS Upgrade Log
5. When both Android OS and Linux OS have been upgraded, restart the device (execute reboot in the Yocto shell environment)

Expected Results:

1. Android OS & Linux OS have both been upgraded, and the upgrade success log is printed in update_android_full.txt: [INFO:update_engine_client_android.cc(103)] onPayloadApplicationComplete(ErrorCode::kSuccess (0))
The log of the successful upgrade printed in update_yocto_full.txt: [INFO:update_attempter_android.cc(600)] Update successfully applied, waiting to reboot.
2. If the upgrade success log is not printed in update_android_full.txt, please execute `adb -s 0123456789ABCDEF pull data/misc/update_engine_log` to capture the update_engine log for RD analysis.
3. If the upgrade is successful, please restart the device, and the first restart after the upgrade should successfully enter the Home Screen.
4. Check whether the system version number has been updated after a successful restart.

Note: If the upgrade fails, please provide update_android_full.txt, update_engine_log and update_yocto_full.txt to the corresponding OTA owner; if it is a reboot failure after the upgrade, please also provide a serial log of the reboot after the upgrade.

4.6.2 Incremental Package Upgrade (Normal Mode)

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get OTA package (including Android OS package and Linux OS package)

Testing Steps:

Note: Please complete the Android OS upgrade first, and then perform the Linux OS upgrade to avoid system startup failure caused by mismatched slots of each OS when the platform powers off and restarts after completing an OS upgrade.

Update Android OS:

1. The device connects to the adb environment, defaulting to connect to Android adb.
2. Execute the following command to connect to Yocto adb
 - `adb forward tcp:7777 tcp:6666`
 - `adb connect 127.0.0.1:7777`
3. Execute adb devices to check whether adb is connected successfully. The correct connection is as follows:
List of devices attached
0123456789ABCDEF device >> Android device
127.0.0.1:7777 device >> Yocto device
4. Execute the Android OS upgrade script: `python3 hypervisor_android_update.py -s 0123456789ABCDEF --file otapackage_android_delta.zip > update_android_delta.txt 2>&1`
 - hypervisor_android_update.py: Android OS upgrade script
 - 0123456789ABCDEF: Android OS device adb number
 - otapackage_android_delta.zip: Android OS OTA full upgrade package, no restrictions on the upgrade package name
 - update_android_delta.txt: Android OS upgrade log

Update Linux OS:

1. The device connects to the adb environment, defaulting to connect to Android adb.
2. Check whether Linux adb is connected; if not connected, please execute the following command to connect yocto SOS adb.
 - `adb forward tcp:7777 tcp:6666`
 - `adb connect 127.0.0.1:7777`
3. Execute adb devices to check whether adb is connected successfully. The correct connection is as follows:
List of devices attached
0123456789ABCDEF device >> Android device
127.0.0.1:7777 device >> Yocto device
4. Execute the Linux OS upgrade script: `python3 hypervisor_yocto_update.py -s 127.0.0.1:7777 --file otapackage_yocto_delta.zip > update_yocto_delta.txt 2>&1`
 - `hypervisor_yocto_update.py`: Linux OS upgrade script
 - `127.0.0.1:7777`: Linux OS device adb number
 - `otapackage_yocto_delta.zip`: Linux OS OTA full upgrade package, no restrictions on the upgrade package name
 - `update_yocto_delta.txt`: Linux OS Upgrade Log
5. When both Android OS and Linux OS have been upgraded, restart the device (execute reboot in the Yocto shell environment)

Expected Results:

1. Android OS & Linux OS have both been upgraded, and the upgrade success log is printed in `update_android_delta.txt`:
[INFO:update_engine_client_android.cc(103)] onPayloadApplicationComplete(ErrorCode::kSuccess (0))
`update_yocto_delta.txt` prints the upgrade success log: [INFO:update_attempter_android.cc(600)] Update successfully applied, waiting to reboot.
2. If the upgrade success log is not printed in `update_android_full.txt`, please execute `adb -s 0123456789ABCDEF pull data/misc/update_engine_log` to capture the `update_engine` log for RD analysis.
3. If the upgrade is successful, please restart the device, and the first restart after the upgrade should successfully enter the Home Screen.
4. Check whether the system version number has been updated after a successful restart.

Note: If the upgrade fails, please provide `update_android_full.txt`, `update_engine_log` and `update_yocto_full.txt` to the corresponding OTA owner; if it is a failure after rebooting post-upgrade, please also provide a serial log of the reboot after the upgrade.

4.7 How to Modify the Signature of the Hypervisor Linux OS (L+L) Upgrade Package

During the OTA upgrade process, two keys will be used: `xxx.pk8` and `xxx.x509.pem`. The key in `xxx.pk8` format is used to sign the upgrade package during the compilation of the upgrade package, while the key in `xxx.x509.pem` format is used to verify the upgrade package during the OTA upgrade process.

If you need to use a customized key, please follow the steps below:

1. Replace the specified path of the `-k` parameter in the upgrade package compilation instruction in [Section 4.4.1.2 Compile Linux OS OTA Upgrade Package](#) with the actual `xxx.pk8` and `xxx.x509.pem` paths used.
2. Replace the actual `xxx.x509.pem` file in `meta/meta-mediatek/recipes-support/update-engine-sideload-u/files/ota/otacerts.zip`.

4.8 How to Remove/Add OTA Upgrade Partitions

4.8.1 Source of OTA Upgrade Partition

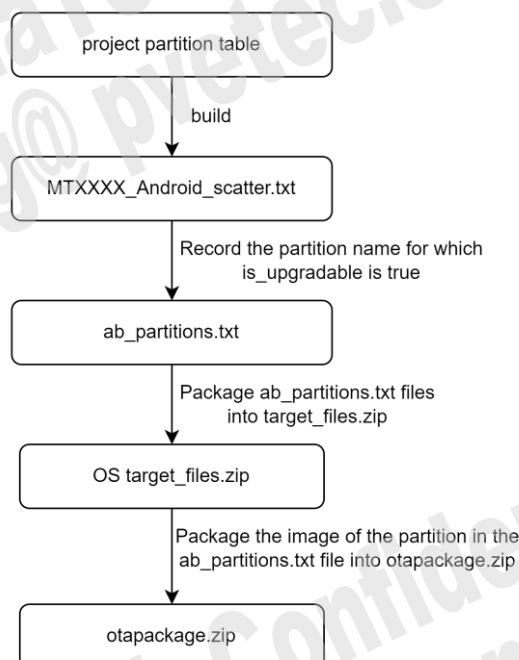


Figure 4-4. OTA upgrade partition source

The confirmation steps for the OTA upgrade partition are as follows:

1. During the compilation phase, the information in the partition table is packaged and converted into MTXXX_Android_scatter.txt, for example: MT6897_Android_scatter.txt.
2. Record the partitions in MT6897_Android_scatter.txt where is_upgradable is true to ab_partitions.txt;
3. Package ab_partitions.txt into the target_files.zip of each OS.
4. When compiling the upgrade package, the partition images recorded in ab_partitions.txt are packaged into the upgrade package.

Note: When checking the is_upgradable status of the partition, the is_upgradable of the _a partition will be prioritized. For example, when the is_upgradable of xxxx_a partition is true and the is_upgradable of xxx_b is false, the system will still record that partition as an OTA upgrade partition.

4.8.2 Remove OTA Upgrade Partition

Note: Reducing the OTA upgrade partition will only disable the OTA upgrade for specific AB partitions and will not change the disabled AB upgrade partition to a single partition.

The Hypervisor (L+L+A) includes three different OS projects, and its image composition includes: Yocto partition image + Android partition image + T-box partition image. The Hypervisor (L+L+A) also includes two partition tables; one partition table is located in the Yocto codebase, and the partition table contains all partition information for Yocto + Android + T-box. This partition table is used during the compilation of Yocto and T-box stages; the other partition table is located in the Android codebase, and the partition table only contains Android partition information. This partition table is only used during the compilation of the Android stage.

- **Yocto & T-box partition table:**
meta/meta-mediatek-mt8676-hyp/recipes-bsp/ptgen/files/<project_name>/partition_table_emmc_hyp_ab.csv
- **Android partition table:**
device/mediateksample/<project_name>/partition_table_emmc_hyp_ab.csv

The following are the methods for reducing OTA upgrade partitions for each OS:

1. **Yocto OS (Host OS)**

Change the OTA_Update status from Y to N in the Yocto partition table to disable the OTA upgrade partition.

For example:

- **Before modification:**

preloader_a	Raw data	1024	EMMC_BOOT1	UFS_LU0	N	Y	bl2.img	Y	N	BOOTLOADERS	Y
preloader_b	Raw data	1024	EMMC_BOOT2	UFS_LU1	N	Y	bl2.img	Y	N	BOOTLOADERS	Y

- **After modification:**

preloader_a	Raw data	1024	EMMC_BOOT1	UFS_LU0	N	Y	bl2.img	N	N	BOOTLOADERS	Y
preloader_b	Raw data	1024	EMMC_BOOT2	UFS_LU1	N	Y	bl2.img	N	N	BOOTLOADERS	Y

2. **Android OS (Guest OS)**

Change the OTA_Update status in the Android & Yocto partition table from Y to N to disable the OTA upgrade partition.

For example:

- **Before modification:**

init_boot_a	Raw data	8192	EMMC_USER	UFS_LU2	N	Y	init_boot.img	Y	N	AUTO	Y
init_boot_b	Raw data	8192	EMMC_USER	UFS_LU2	N	N	NONE	N	N	AUTO	Y

- **After modification:**

init_boot_a	Raw data	8192	EMMC_USER	UFS_LU2	N	Y	init_boot.img	N	N	AUTO	Y
init_boot_b	Raw data	8192	EMMC_USER	UFS_LU2	N	N	NONE	N	N	AUTO	Y

3. **Tbox OS (Guest OS)**

Change the OTA_Update status from Y to N in the Tbox partition table for the section that requires disabling OTA upgrade.

For example:

- **Before modification:**

bl2-tbox_a	Raw data	1024	EMMC_USER	UFS_LU2	N	Y	bl2-tbox.img	Y	N	AUTO	Y
bl2-tbox_b	Raw data	1024	EMMC_USER	UFS_LU2	N	N	NONE	Y	N	AUTO	Y

- **After modification:**

bl2-tbox_a	Raw data	1024	EMMC_USER	UFS_LU2	N	Y	bl2-tbox.img	N	N	AUTO	Y
bl2-tbox_b	Raw data	1024	EMMC_USER	UFS_LU2	N	N	NONE	N	N	AUTO	Y

4.8.3 **OTA Upgrade Partition**

The newly added OTA upgrade partition must meet the following prerequisites:

1. The partition must simultaneously contain <partition_name>_a & <partition_name>_b, for example: boot_a & boot_b.
2. The partition must be a read-only partition.
3. The partition image format and image name must be <partition_name>.img, for example: boot.img.

4.8.3.1 Set the Existing AB Partition as the OTA Partition

The steps for each OS to set the existing AB partitions in the partition table as OTA partitions are as follows:

1. Yocto OS (Host OS)

Change the OTA_Update status from N to Y in the Yocto partition table for the OTA upgrade section.

For example:

- Before modification:

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,b12.img,N,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,b12.img,N,N,BOOTLOADERS,Y
```

- After modification:

```
preloader_a,Raw data,1024,EMMC_BOOT1,UFS_LU0,N,Y,b12.img,Y,N,BOOTLOADERS,Y
preloader_b,Raw data,1024,EMMC_BOOT2,UFS_LU1,N,Y,b12.img,Y,N,BOOTLOADERS,Y
```

2. Android OS (Guest OS)

Change the OTA_Update status from N to Y in the Yocto partition table and Android partition table under the OTA upgrade section.

For example:

- Before modification:

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,N,N,AUTO,Y
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

- After modification:

```
init_boot_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,init_boot.img,Y,N,AUTO,Y
init_boot_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new Android OTA partition name to the ignored_partitions in Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip, preventing errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'tee', 'b12-tbox', 'boot_uos_tbox'} >> Add the
newly added partitions to the list
```

3. T-box OS (Guest OS)

Change the OTA_Update status from N to Y in the T-box partition table for the section that requires disabling OTA upgrade.

For example:

- Before modification:

```
b12-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,b12-tbox.img,N,N,AUTO,Y
```

```
b12-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,N,N,AUTO,Y
```

- After modification:

```
b12-tbox_a,Raw data,1024,EMMC_USER,UFS_LU2,N,Y,b12-tbox.img,Y,N,AUTO,Y
```

```
b12-tbox_b,Raw data,1024,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new T-box OTA partition name to the `ignored_partitions` in `Hypervisor_make_yocto_targetfiles.py`. This addition allows skipping this partition during the compilation of `yocto_target_files.zip` to prevent errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmeta', 'vbmeta_system', 'vbmeta_vendor', 'tee', 'b12-tbox', 'boot_uos_tbox'} >> Add the
newly added partitions to the list
```

Add the new T-box OTA partition name in the partitions of `Hypervisor_make_uos_tbox_targetfiles.py`. Ensure that the T-box compilation stage packages the partition image into `uos_tbox_target_files.zip`.

For example:

```
if __name__ == "__main__":
    target_path = "sos_tbox_images"
    partitions = ['b12-tbox', 'boot_uos_tbox'] >> Add the newly added partition to the
list
```

4.8.3.2 Add a Non-existent AB Partition as an OTA Partition

The steps to add the OTA partition for the AB partition that does not exist in the partition table of each OS are as follows:

1. Yocto OS (Host OS)

Add the corresponding AB partition in the Yocto partition table, and ensure that the partition image is `<partition_name>.img`, taking the addition of the customer partition as an example:

- After modification (the partition size is set to 8MB):

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
```

```
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

After completing the partition addition, add logic to determine the AB partition during the boot loading phase of the customer partition to ensure that the system loads into the correct partition.

2. Android OS (Guest OS)

Add the corresponding AB partitions in the Yocto partition table and Android partition table, and ensure that the partition image is `<partition_name>.img`, taking the addition of the customer partition as an example:

- After modification (the partition size is set to 8MB):

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
```

```
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new Android OTA partition name to the `ignored_partitions` in

Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip, preventing errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmata', 'vbmata_system', 'vbmata_vendor', 'customer', 'tee', 'bl2-tbox',
'boot_uos_tbox'} >> Add the newly added partitions to the list
```

3. T-box OS (Guest OS)

Add the corresponding AB partition in the Yocto partition table, and ensure that the partition image is <partition_name>.img, taking the addition of the customer partition as an example:

- After modification (the partition size is set to 8MB):

```
customer_a,Raw data,8192,EMMC_USER,UFS_LU2,N,Y,customer.img,Y,N,AUTO,Y
customer_b,Raw data,8192,EMMC_USER,UFS_LU2,N,N,NONE,Y,N,AUTO,Y
```

Add the new Android OTA partition name to the ignored_partitions in Hypervisor_make_yocto_targetfiles.py. This addition allows skipping the partition during the compilation of yocto_target_files.zip, preventing errors related to non-existent partition images during the compilation phase.

For example:

```
ignored_partitions = {'dtbo', 'boot', 'vendor_boot', 'init_boot', 'userdata', 'super',
'vbmata', 'vbmata_system', 'vbmata_vendor', 'tee', 'bl2-tbox', 'boot_uos_tbox',
'customer'} >> Add the newly added partitions to the list
```

Add the new T-box OTA partition name in the partitions of Hypervisor_make_uos_tbox_targetfiles.py. Ensure that the T-box compilation stage packages the partition image into uos_tbox_target_files.zip.

For example:

```
if __name__ == "__main__":
    target_path = "sos_tbox_images"
    partitions = ['bl2-tbox', 'boot_uos_tbox', 'customer'] >>Add the newly added partition
to the list
```

Note:

1. When adding a non-existent AB partition as an OTA partition, please ensure that the newly added partition is mounted as a read-only partition.
2. When adding a non-existent AB partition as an OTA partition, please ensure that the system can load and start normally after the new partition is added, and then verify the OTA upgrade.

4.9 Selection of Boot Slot

4.9.1 Boot Control Information Description

Due to the independent upgrades of Linux OS and Android OS in the Hypervisor (L+L+A) partial centralized upgrade scheme, in order to facilitate the management of each OS's boot slot, the boot control parameters of Hypervisor (L+L+A) also include two independent copies, which are used to control the boot slot selection for Linux OS and Android OS respectively. The storage locations and key information of each boot control parameter are as follows:

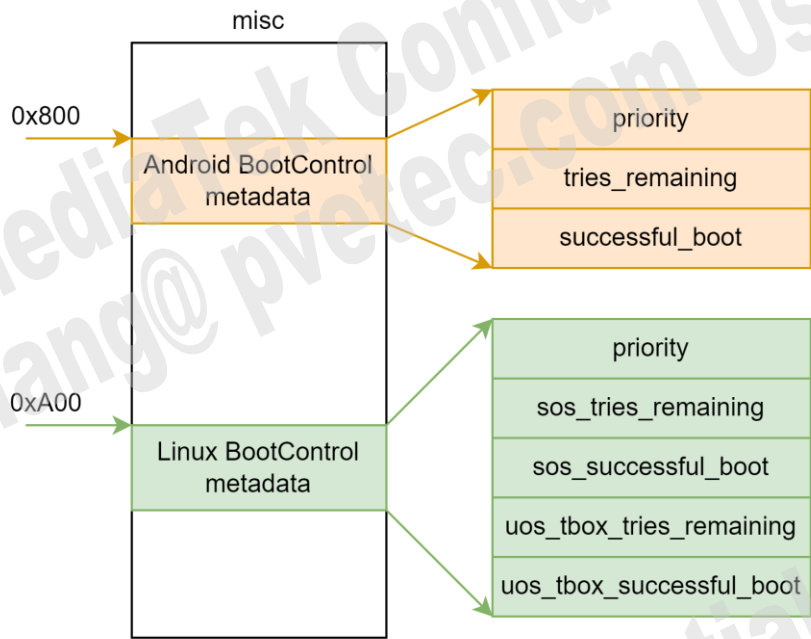


Figure 4-5. Hypervisor (L+L+A) boot control parameters

The description of each boot control parameter in Figure 4-5 is shown in Table 4-1:

Table 4-1. Boot control parameter description

System type	Name	Description
Android	priority	Indicates the priority of the Android UOS Slot, with an initial value of 15. The system defaults to selecting the slot with a higher priority to start.
	tries_remaining	Indicates the number of reboot attempts for the Android UOS Android slot, the default value is 7, and 0 means that a rollback needs to be triggered.
	successful_boot	Android UOS startup success flag, 1: Startup successful, 0: Startup failed
Linux	priority	Indicates the priority of the Linux Slot, with an initial value of 15. The system defaults to selecting the slot with a higher priority to start.
	sos_tries_remaining	Indicates the number of reboot attempts for the Linux SOS slot, the default value is 7, and 0 means that a rollback needs to be triggered.
	sos_successful_boot	Linux SOS startup success flag, 1: Startup successful, 0: Startup failed
	uos_tbox_tries_remaining	Indicates the number of restart times for the T-box UOS slot, the default value is 7, and 0 means that a rollback needs to be triggered.
	uos_tbox_successful_boot	Linux T-box UOS startup success flag, 1: Success, 0: Failure

Note: Boot Control Source Code is located at: `src/bsp/lk2/platform/mediatek/common/bootctrl/bootctrl_v1`

4.9.2 Boot Slot Selection Mechanism

Since Hypervisor (L+L+A) has two independent boot control information, in order to avoid inconsistent boot slots for Linux OS and Android OS, Hypervisor (L+L+A) adds a boot slot selection mechanism in Linux SOS LK2 to ensure that Linux OS and Android OS always start from the same slot. The flow of the selection mechanism is shown in Figure 4-6:

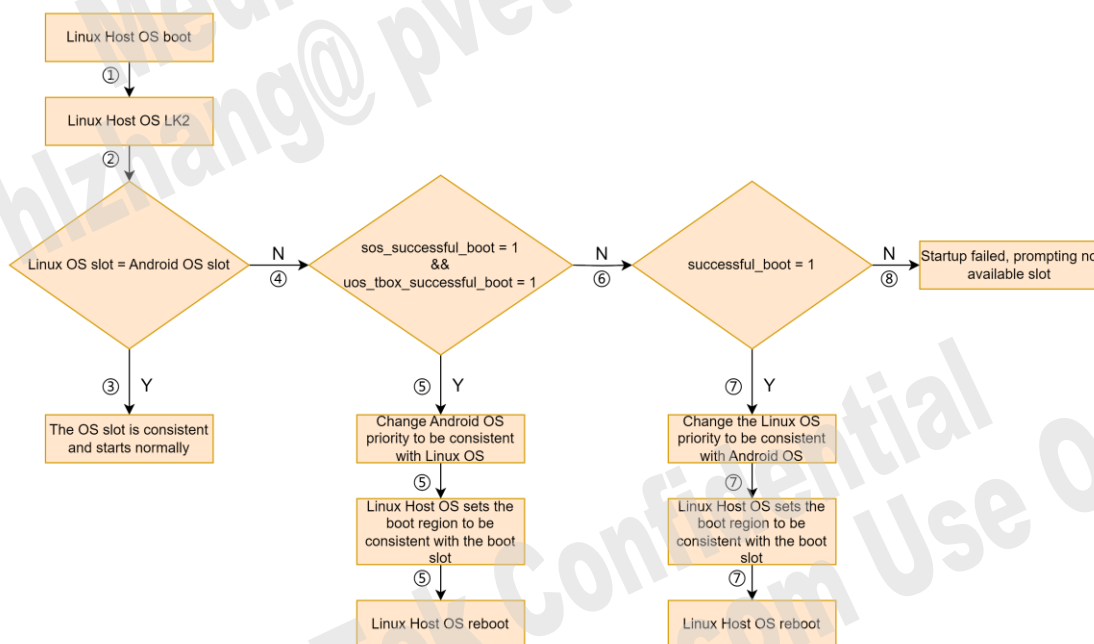


Figure 4-6. Hypervisor (L+L+A) boot slot selection mechanism

The boot slot selection mechanism flow shown in Figure 4-6 is described as follows:

- ① Start Linux Host OS.
- ② Linux Host OS enters LK2 stage, checks whether the current boot slot of Linux OS and Android OS is consistent.
- ③ If the current boot slot of the Linux OS is consistent with that of the Android OS, then boot directly from that slot.
- ④ If the current boot slot of the Linux OS is inconsistent with that of the Android OS, then continue to check whether the `successful_boot` of both the Linux Host OS and the Linux Guest OS under the current Linux OS boot slot are 1.
- ⑤ If the `successful_boot` of both the Linux Host OS and Linux Guest OS under the current Linux OS boot slot is 1, then modify the priority of Android OS to be consistent with the Linux OS, set the boot region to be consistent with the current Linux OS boot slot, and finally restart the Linux SOS to ensure that the Linux OS and Android OS boot slots are consistent.
- ⑥ If the `successful_boot` of both the Linux Host OS and the Linux Guest OS under the current Linux OS boot slot is not 1, then continue to check whether the `successful_boot` of the current Android OS is 1.
- ⑦ If the current Android OS's `successful_boot` is 1, then modify the Linux OS's priority to be consistent with the Android OS, set the boot region to be consistent with the current Android OS boot slot, and finally restart the Linux SOS to ensure that the Linux OS and Android OS boot slots are consistent.
- ⑧ If all OS's `success_boot` values are not 1, it is determined that the current boot slot is unavailable, and the system startup fails.

Note: The source code related to Rollback is located at: `src/bsp/lk2/platform/mediatek/common/bootctrl/bootctrl_v1`

4.10 Android Merge

The Hypervisor (L+L+A) Android OS supports Virtual A/B. After the Android OS upgrade completes and successfully boots from the new slot for the first time, the Android OS merges the new slot super partition image stored in either the super partition or userdata partition to overwrite the old slot super partition. Image data indicates that only one available super image of Android OS exists at this time. Therefore, before initiating the Android OS merge flow, ensure that all Linux OSs (OS) and Android OSs (OS) have successfully started. This prevents scenarios where a failure in starting one OS after the Android OS merge triggers the rollback mechanism, resulting in a failed rollback of the Android OS.

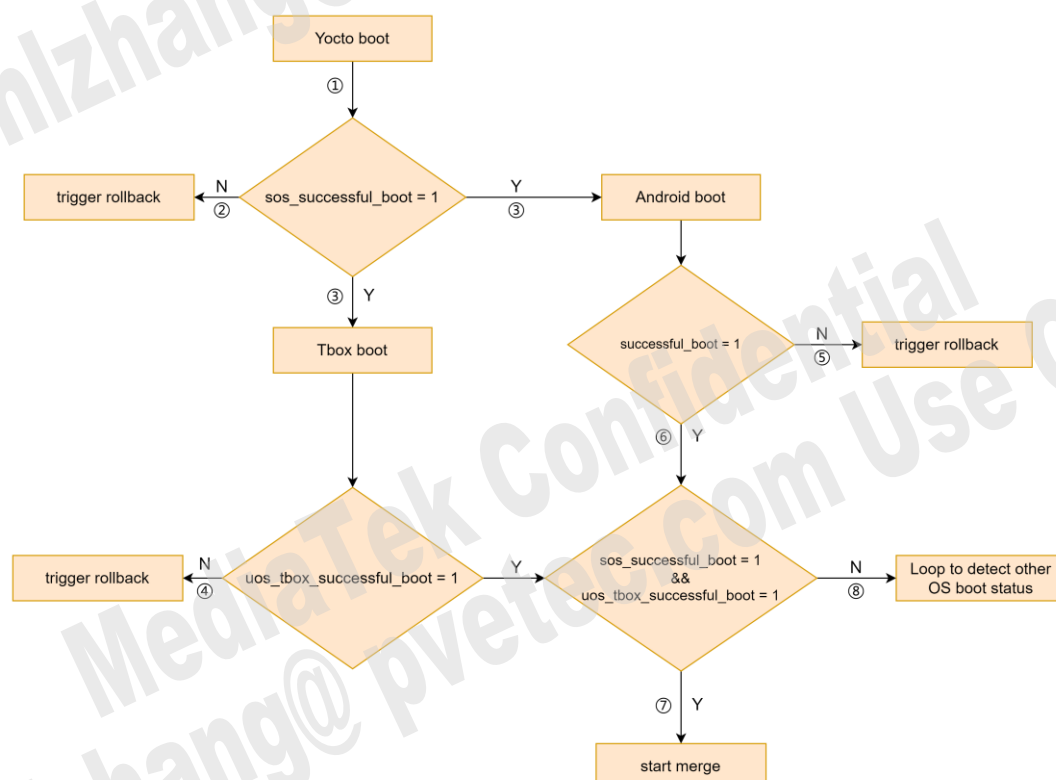


Figure 4-7. Android merge mechanism

Figure 4-7 describes the prerequisites for Android OS to start the merge. The process is as follows:

- ① Start Yocto OS, check whether Yocto OS has successfully booted, that is, whether `sos_successful_boot` has been set to 1.
- ② If `sos_successful_boot` is not set to 1 before the value of `sos_tries_remaining` decreases to 0, then Yocto OS is determined to have failed to boot, triggering the rollback mechanism.
- ③ If Yocto OS starts successfully, the system will set `sos_successful_boot` to 1 and start Android OS and T-box OS.
- ④ If `uos_tbox_successful_boot` is not set to 1 before the value of `uos_tbox_tries_remaining` decreases to 0, then T-box OS is deemed to have failed to start, triggering the rollback mechanism.
- ⑤ If `successful_boot` is not set to 1 before the `tries_remaining` value decreases to 0, then the Android OS is deemed to have failed to boot, triggering the rollback mechanism.
- ⑥ If Android UOS starts successfully, the value of `successful_boot` is set to 1. At this time, Android UOS will monitor the startup status of Yocto OS and Tbox OS.
- ⑦ If Yocto OS and T-box OS's `successful_boot` are both set to 1, which means boot is successful, Android OS will then start the merge process.

⑧ If both Yocto OS and T-box OS's successful_boot are not 1, then Android OS will monitor in a loop.

4.11 System Rollback

This section primarily describes how the Hypervisor (L+L+A) handles system rollback to the previous version after an OTA upgrade fails during the first boot from the new version, preventing device bricking.

4.11.1 Basic Principles of System Rollback

Since the Hypervisor (L+L+A) Project involves multiple OSs, and the Android OS needs to support Virtual AB, the rollback adopted by MediaTek follows the principles below:

- Linux OS & Android OS overall rollback, that is, when any one OS fails to start from the first new version, all OSs must roll back to the old system startup to ensure that all OSs' AB slots are consistent.
- Only in the scenario where the first boot from the new system fails after a successful OTA upgrade will the rollback mechanism be triggered; regular boot failures will not trigger the rollback mechanism.

Taking Yocto, T-box, and Android 3 OS (L+L+A) OTA as an example, the system will have the following three rollback scenarios:

Scenario 1: Yocto OS failed to boot from the new version

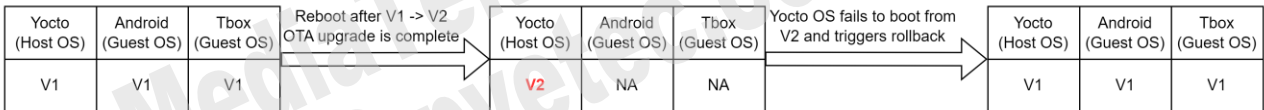


Figure 4-8. Yocto boot failure scenario after OTA upgrade

Figure 4-8 describes the version switching status of Yocto, T-box, and Android after the Hypervisor (L+L+A) OTA upgrade from V1 -> V2 is completed. When Yocto OS fails to start from the new version V2 and triggers a rollback, the scenario is described as follows:

- Yocto, Android, and T-box all start from version V1.
- The system executes an OTA upgrade to version V2 and sets version V2 as the version for the next startup.
- Yocto fails to start from version V2; at this time, Android and T-box have not yet been initiated.
- Trigger rollback. Yocto, Android, and T-box switch to V1 version for startup.

Scenario 2: Android OS fails to boot from the new version



Figure 4-9. Android boot failure scenario after OTA upgrade

Figure 4-9 describes the version switching status of Yocto, Android, and T-box after the Hypervisor (L+L+A) is upgraded from V1 -> V2 OTA. When Yocto OS successfully boots from the new version V2 and Android OS fails to boot from the new version V2, triggering a rollback, the scenario is described as follows:

- Yocto, Android, and T-box all start from version V1.
- The system executes an OTA upgrade to version V2 and sets version V2 as the next boot version.
- Yocto successfully started from version 2. Android failed to start from version 2. The T-box startup status is unknown.
- Trigger rollback. Yocto, Android, and T-box switch to V1 version for startup.

Scenario 3: T-box OS failed to boot from the new version

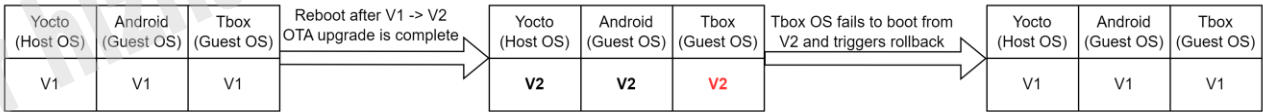


Figure 4-10. T-box boot failure scenario after OTA upgrade

Figure 4-10 describes the version switching status of Yocto, Android, and T-box after the Hypervisor (L+L+A) is upgraded from V1 -> V2 OTA. When Yocto OS and Android OS successfully boot from the new version V2, and T-box OS fails to boot from the new version, triggering a rollback. The scenario is described as follows:

- Yocto, Android, and T-box all start from version V1.
- The system executes an OTA upgrade to version V2 and sets version V2 as the version for the next startup.
- Yocto and Android successfully boot from version V2; T-box fails to boot from version V2.
- Trigger rollback. Yocto, Android, and T-box switch to V1 version for startup.

4.11.2 System Rollback Process

The system mechanism of Hypervisor (L+L+A) is set in the LK2 startup phase of each OS. The detailed process is shown in Figure 4-11:

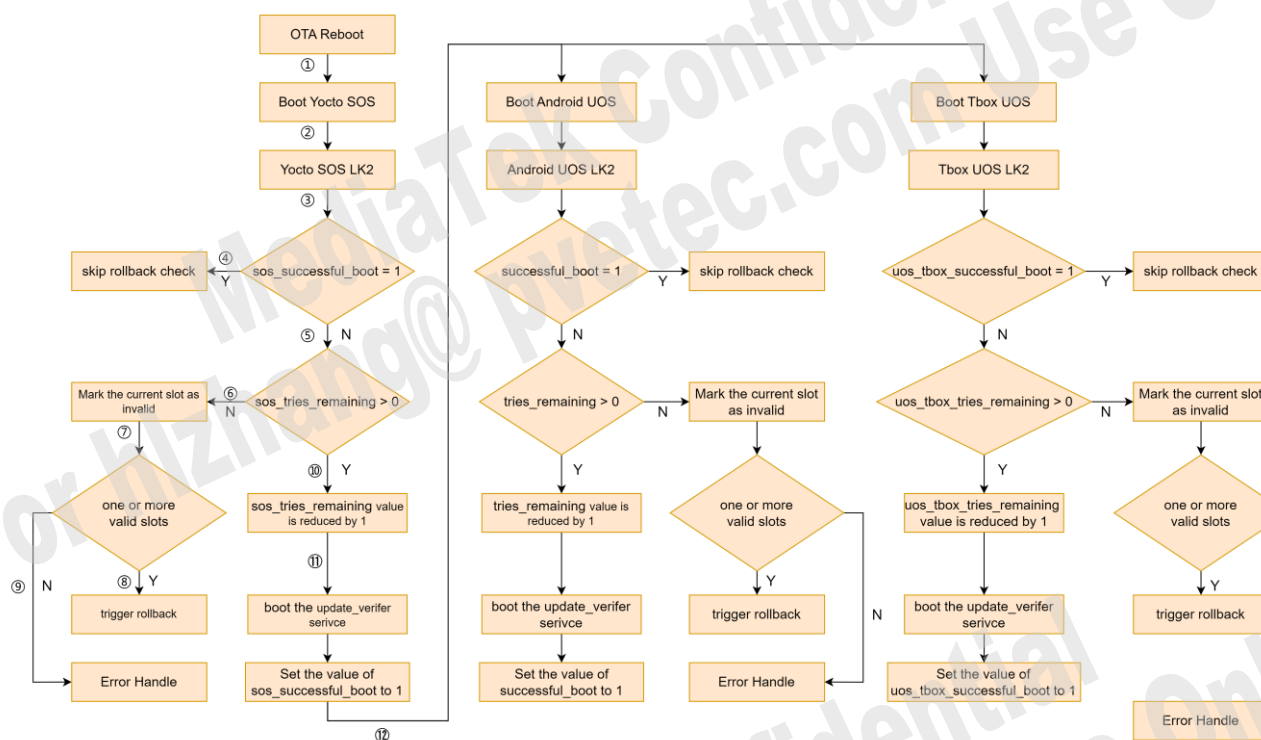


Figure 4-11. Hypervisor rollback flow

4.11.3 System Test Cases

4.11.3.1 Yocto OS Boot Failed

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get the OTA package from the OTA owner

Testing Steps:

1. Refer to [Section 4.6 How to Perform Hypervisor \(L+L+A\) OTA Upgrade](#) to execute the Hypervisor (L+L+A) OTA upgrade, and do not restart the device after the upgrade.
2. Execute the following command in the Yocto OS shell environment:
 - `adb -s 127.0.0.1:7777 shell`
 - `blockdev --setrw /dev/disk/by-partlabel/yocto-boot_b`
 - `dd if=/dev/urandom of=/dev/disk/by-partlabel/yocto-boot_b bs=1M count=1`
 - `reboot`

Expected Results:

1. Yocto SOS system startup failed and triggered the rollback mechanism to successfully start from the pre-upgrade system.
2. The system version number remains consistent with the version before the upgrade.

4.11.3.2 Android OS Boot Failed

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get the OTA package from the OTA owner

Testing Steps:

1. Refer to [Section 4.6 How to Perform Hypervisor \(L+L+A\) OTA Upgrade](#) to perform Hypervisor (L+L+A) OTA upgrade. Do not reboot the device after the upgrade.
2. Execute the following command in the Android SOS environment:
 - adb -s 127.0.0.1:7777 shell
 - blockdev --setrw /dev/disk/by-partlabel/init_boot_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/init_boot_b bs=1M count=1
 - reboot

Expected Results:

1. The Android UOS system failed to start and triggered the rollback mechanism to successfully start from the pre-upgrade system.
2. The system version number remains consistent with the version before the upgrade.

4.11.3.3 T-box OS Boot Failed

Test Environment:

1. The PC has a Python 3 environment
2. Flash base load in the DUT
3. Get the OTA package from the OTA owner

Testing Steps:

1. Refer to [Section 4.6 How to Perform Hypervisor \(L+L+A\) OTA Upgrade](#) to perform Hypervisor (L+L+A) OTA upgrade. Do not reboot the device after the upgrade.
2. Execute the following command in the Android SOS environment:
 - adb -s 127.0.0.1:7777 shell
 - blockdev --setrw /dev/disk/by-partlabel/boot_uos_tbox_b
 - dd if=/dev/urandom of=/dev/disk/by-partlabel/ boot_uos_tbox_b bs=1M count=1
 - reboot

Expected Results:

1. T-box UOS system startup failed and triggered the rollback mechanism to successfully start from the pre-upgrade system.
2. The system version number remains consistent with the version before the upgrade.

4.12 Power Outage Upgrade Protection

Since the device may be interrupted by abnormal power failure during the OTA upgrade process, in order to ensure that the device can continue to complete the upgrade from the interruption point after power failure and restart during the OTA upgrade process, the system needs to save the upgrade status information in real time. The Linux OS upgrade status information is stored in the `data/misc/update_engine/prefs` directory of the yocto-userdata partition of Yocto SOS by default, and the Android OS upgrade status information is stored in the `data/misc/update_engine/prefs` directory of the userdata partition of Android UOS by default.

4.12.1 Upgrade Status Information

Hypervisor (L+L+A) Linux OS and Android OS have consistent upgrade status information saved during the OTA upgrade process, mainly including the following information:

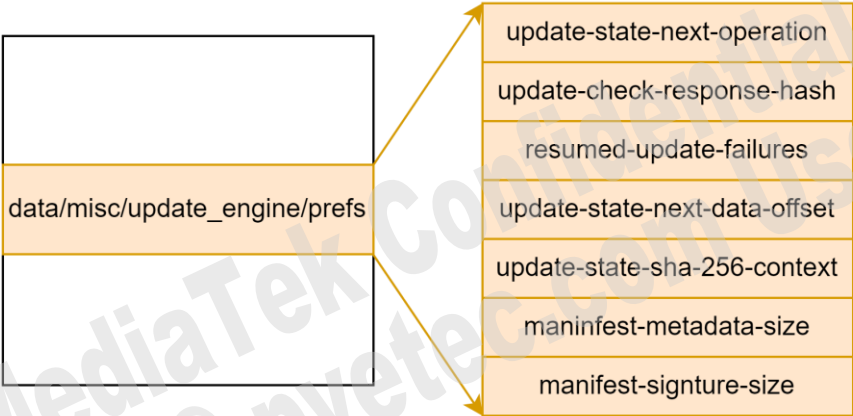


Figure 4-12. Upgrade status information

Table 4-2 describes the functions of each upgrade status information:

Table 4-2. Upgrade status information description

Prefs name	Prefs value	Description
kPrefsUpdateStateNextOperation	update-state-next-operation	Record the next operation that needs to be performed.
kPrefsUpdateCheckResponseHash	update-check-response-hash	Record the hash value of the last upgrade package.
kPrefsResumedUpdateFailures	resumed-update-failures	Record the number of times the upgrade is interrupted; by default, an upgrade can be interrupted a maximum of 10 times. If it exceeds 10 times, the system will force a complete re-upgrade.
kPrefsUpdateStateNextDataOffset	update-state-next-data-offset	Record the offset address of the data downloaded from the upgrade package next time.
kPrefsUpdateStateSHA256Context	update-state-sha-256-context	Record the hash value of the upgrade status information

Prefs name	Prefs value	Description
kPrefsManifestMetadataSize	manifest-metadata-size	Record the size of metadata data
kPrefsManifestSignatureSize	manifest-signature-size	Record signature data size

4.12.2 Resume Upgrade Inspection Process

Please refer to the description in [Section 3.12.2 Resume Upgrade Inspection Process](#).

Attachment 1 Additional Terms

The acquisition, download, and use of this document and its related information by you (or your company or other legal entity collectively referred to as "you") are contingent upon your acceptance of these additional terms. The use, acquisition, or download of this document indicates acceptance of these additional terms and agreement to be bound by them. If there is disagreement with being bound by these additional terms, do not use, acquire, or download this document and immediately delete or destroy all copies of this document.

This document contains confidential and proprietary information of MediaTek Inc. and its affiliates (hereinafter collectively referred to as "MediaTek") or its authorized representatives. This information is intended solely for internal use related to the MediaTek chipset described in this document and must not be used for any other purpose, including but not limited to confirming or providing evidence supporting any potential patent infringement claims against MediaTek, its suppliers, and/or its direct or indirect customers. Unauthorized use or disclosure of this document and the information contained herein is prohibited. Compensation shall be provided to MediaTek for any losses or damages incurred due to unauthorized use or disclosure of any part or all of this document and the information contained herein.

MediaTek and its licensors retain all rights to ownership and composition of this document and do not grant any intellectual property rights (whether express or implied, through estoppel or otherwise). MediaTek may change the content of this document at any time without prior notice. MediaTek assumes no liability related to the use or reliance on this document or any consequences arising from such use or reliance, including but not limited to indirect damages or incidental damages.

This document and any other materials, information, or technical support provided by MediaTek are delivered on an as-is basis. MediaTek assumes no responsibility for any express, implied, statutory, or other forms of warranty, particularly regarding merchantability, non-infringement, suitability for a particular purpose, completeness or accuracy of information, or any warranties arising from trade practices or transactions. MediaTek shall not be liable for any deliverables made to meet specified requirements or comply with specific standards or standard organizations.

Without limiting the aforementioned terms, MediaTek does not assume any warranty or guarantee regarding the applicability of its products for any specific purpose, nor does it bear any liability for damages arising from the application or use of products, circuits, or software. Responsibility for all design, validation, and testing related to the integration of personal products with MediaTek products rests solely with the user. Ensure that the integrated products comply with relevant standards and any safety requirements or other specifications.

This supplementary clause and all actions related to this supplementary clause or this document shall be governed, interpreted, and clarified by the laws of Taiwan, without applying principles of conflict of laws.